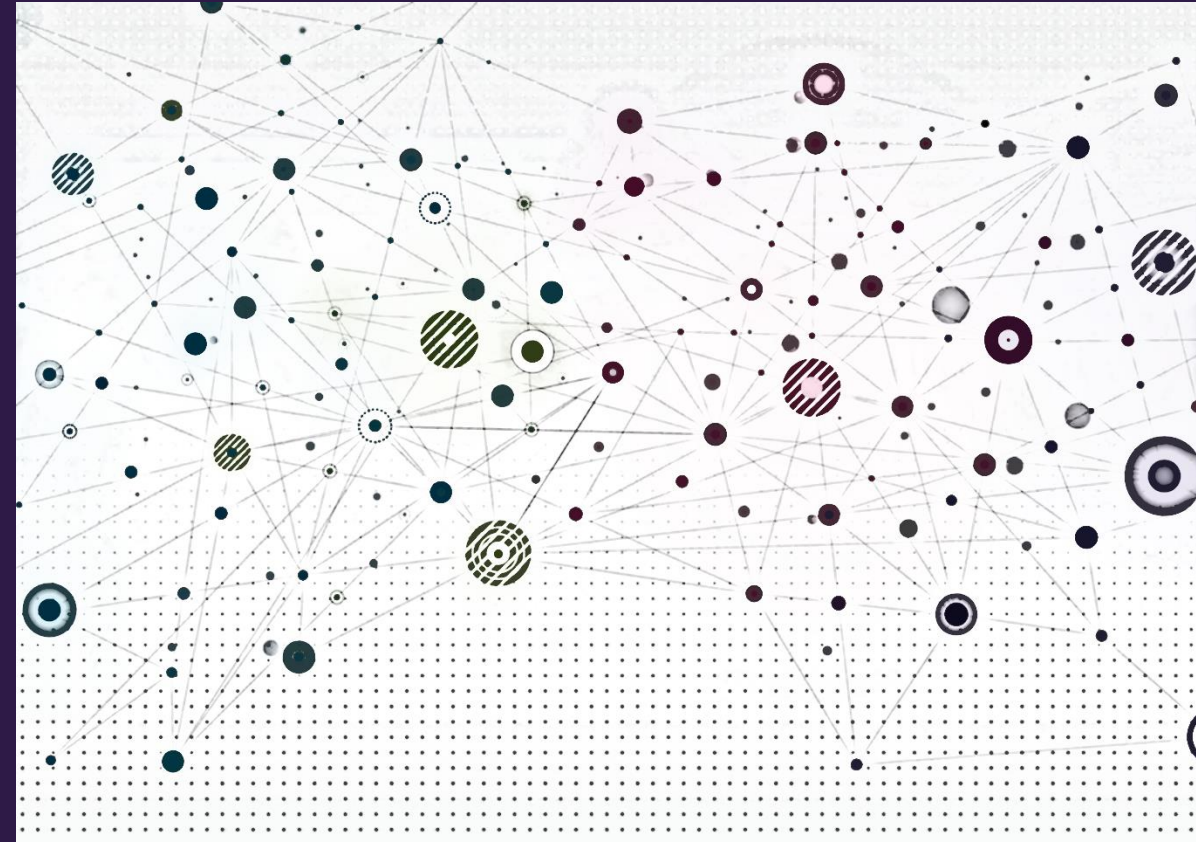


Graph Transformers





Overview

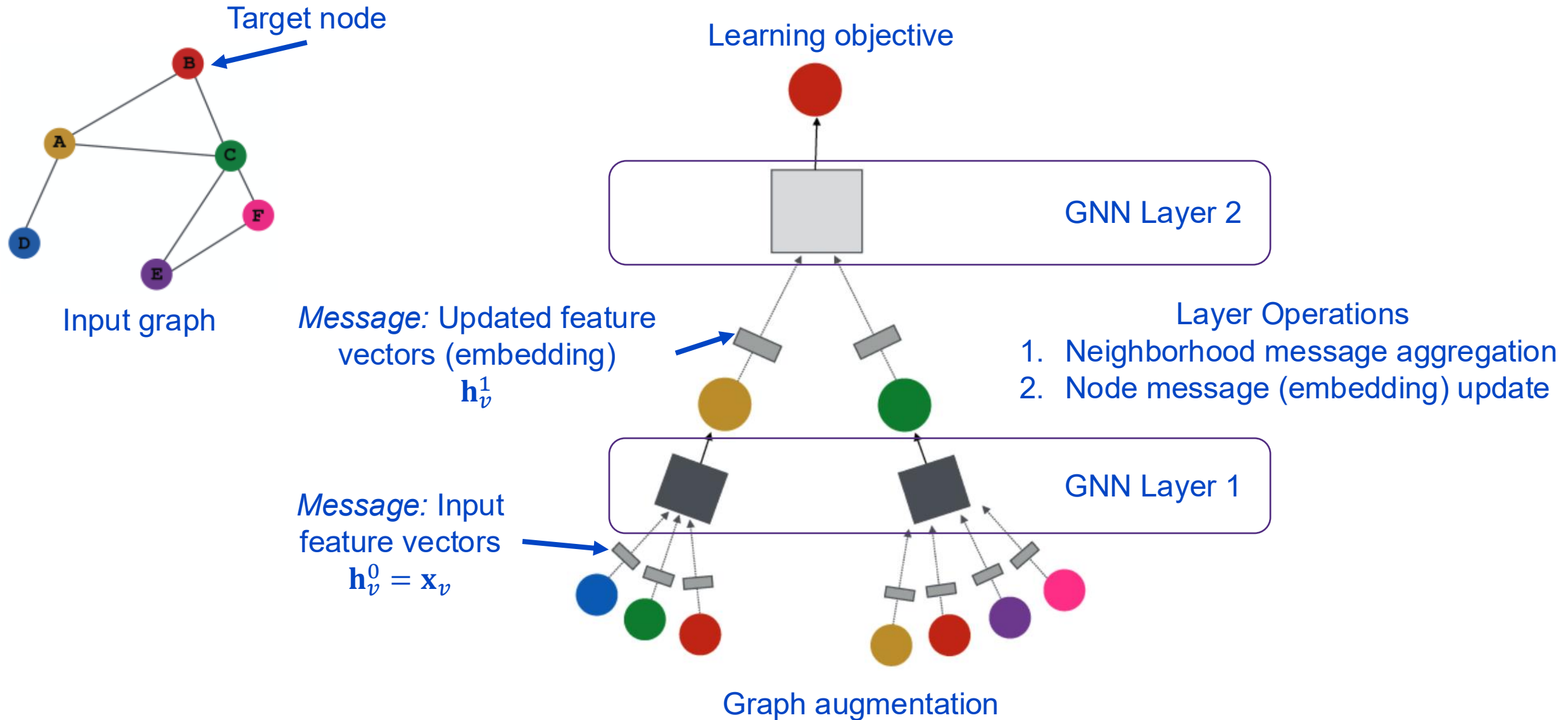
- Recap
- Introducing transformers
- Transformers as a GNN
- Transformers on graphs



Recap

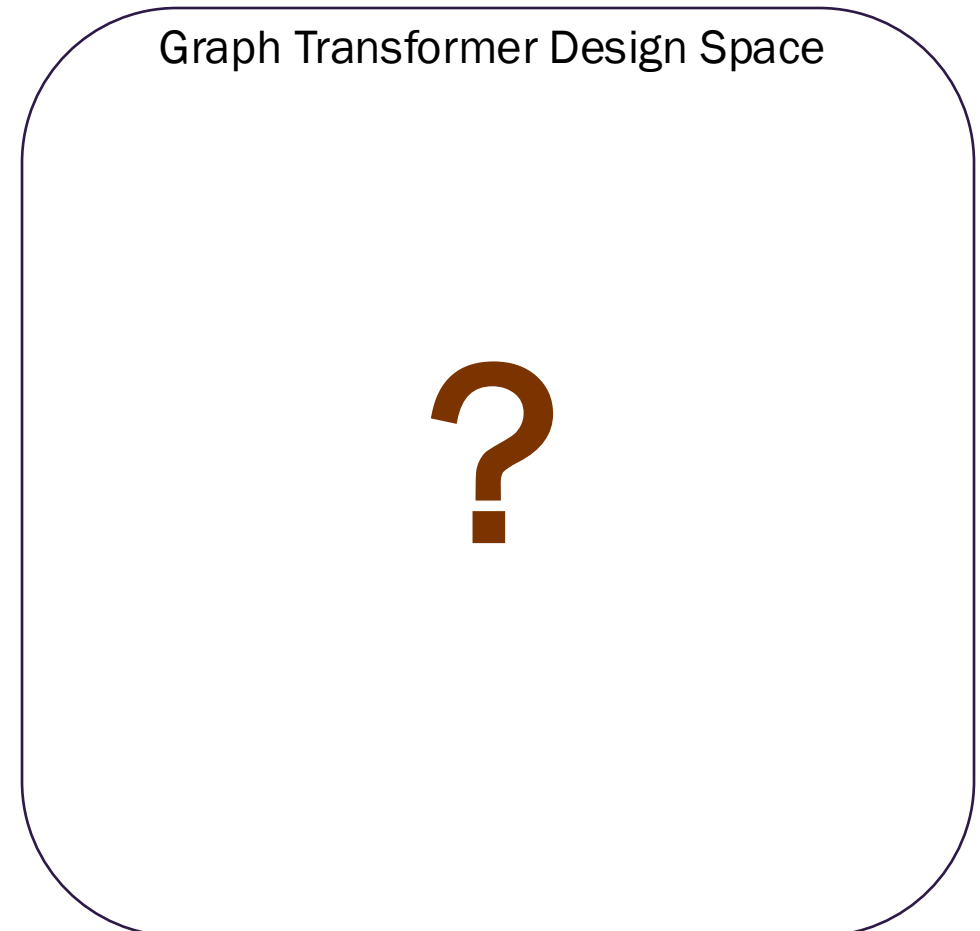
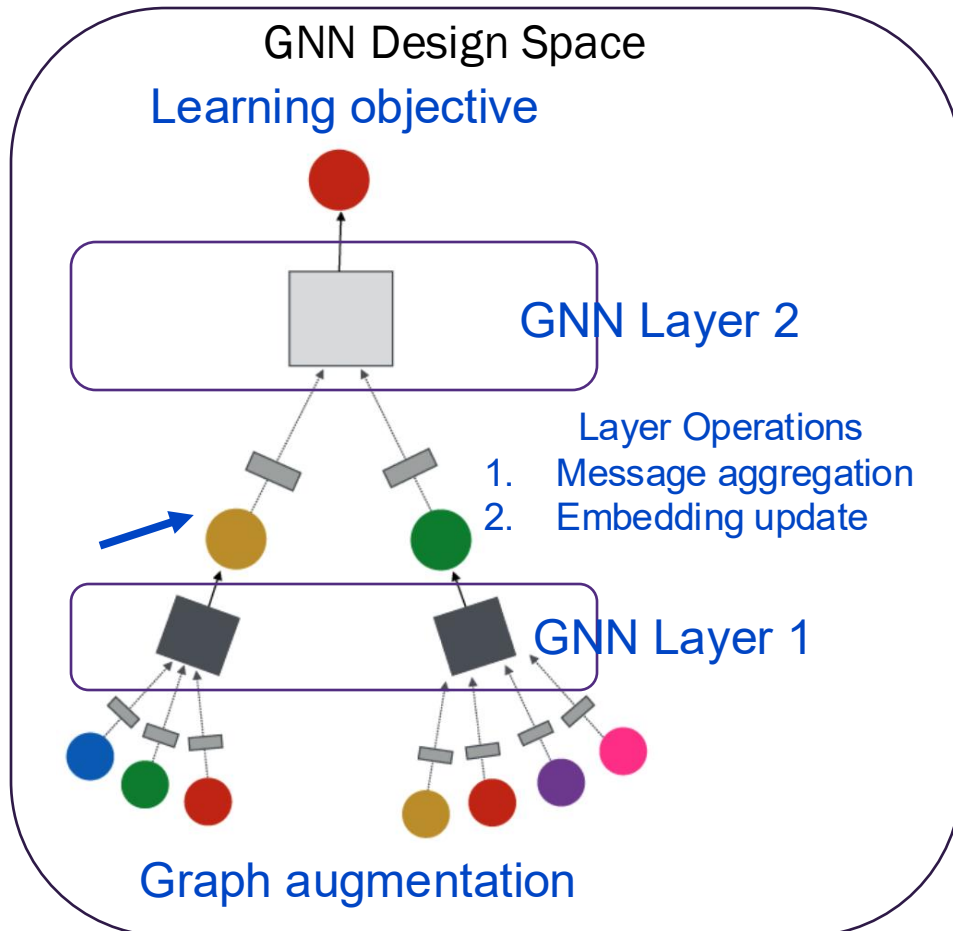


Message passing as a general GNN framework



Message passing as a general GNN framework

- We have covered a lot about the GNN design space
- What can we say about the Graph Transformer design space?



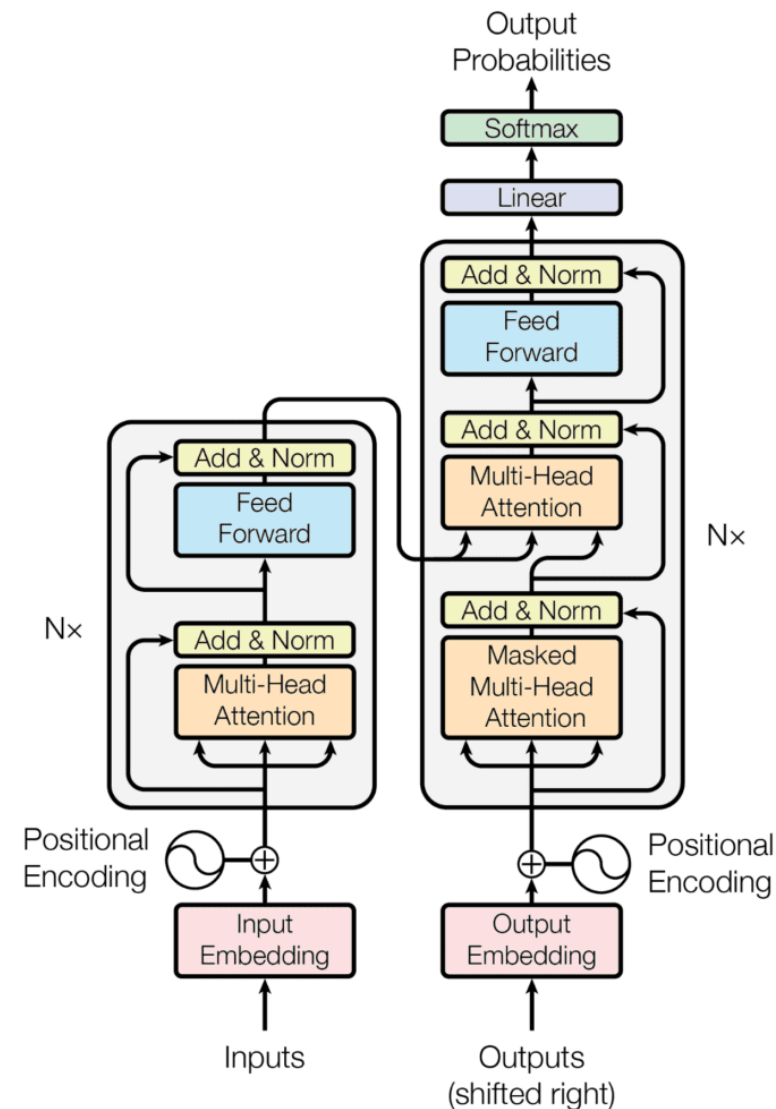


Introducing transformers



Transformers are the foundation of today's generative AI

- Introduced by Vaswani et al. in 2017
- Introduces the concept of *self-attention*
- Includes *encoder and decoder* components
 - Encoder forms a representation of the input
 - Decoder processes that to generate output
- Initial applications were NLP focused. Later applications include computer vision and time series.
- All LLMs / LRMs based on transformers





Leveraging transformers for graphs

There are many multi-billion node / graph scale data

Software execution

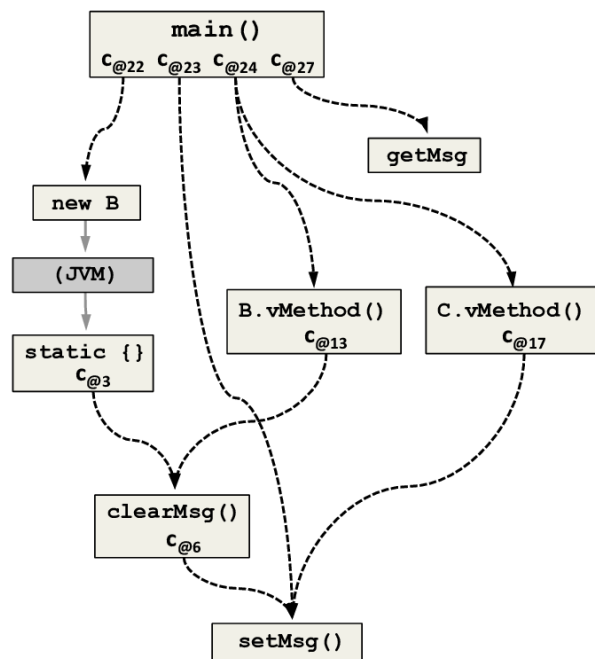


Image credit: [ResearchGate](#)

Knowledge graphs

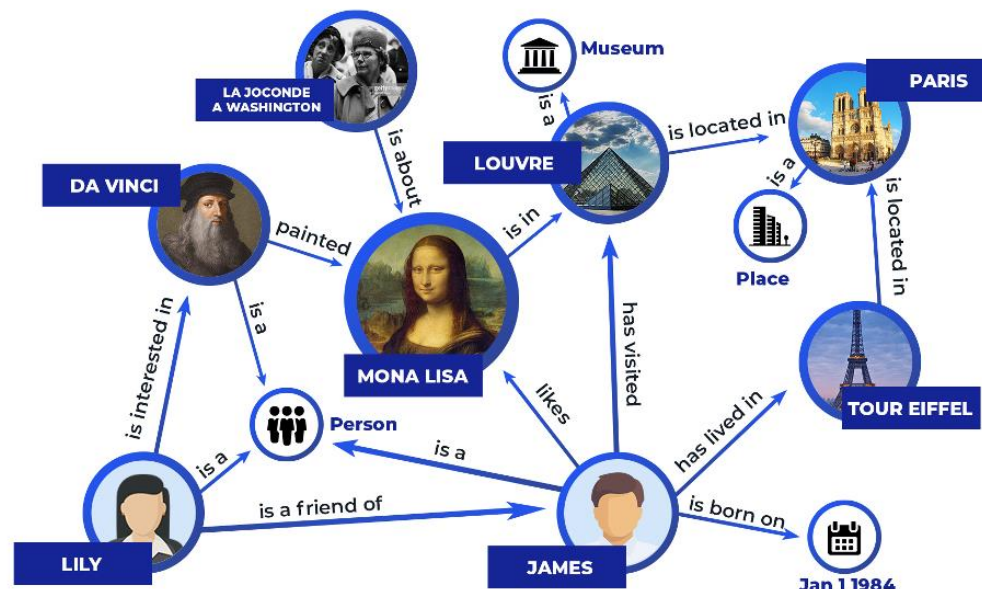


Image credit: [Zilliz](#)

Molecules

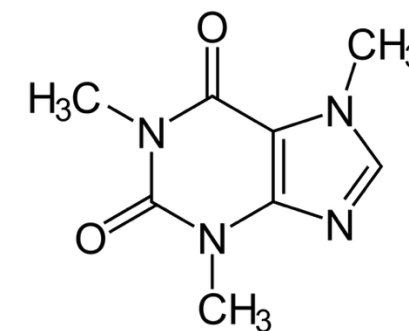
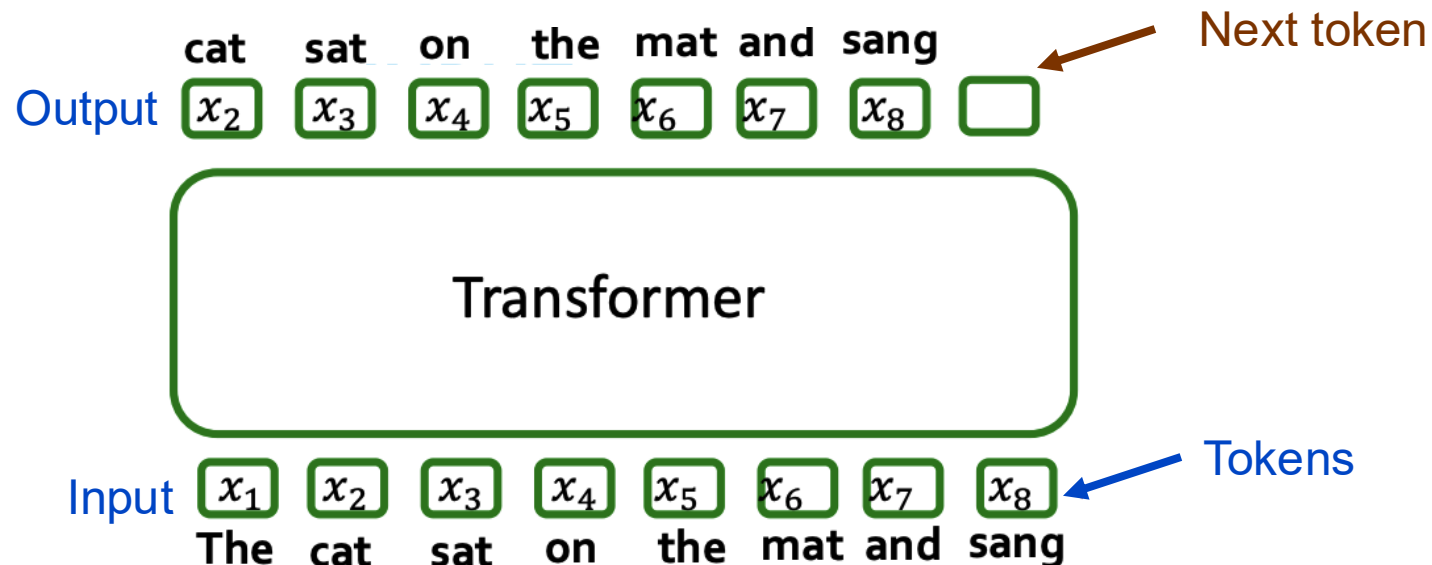


Image credit: [Chemistry StackExchange](#)

Transformers and input tokens

- Transformers map sequences of vectors to sequences of vectors (**encoder**)

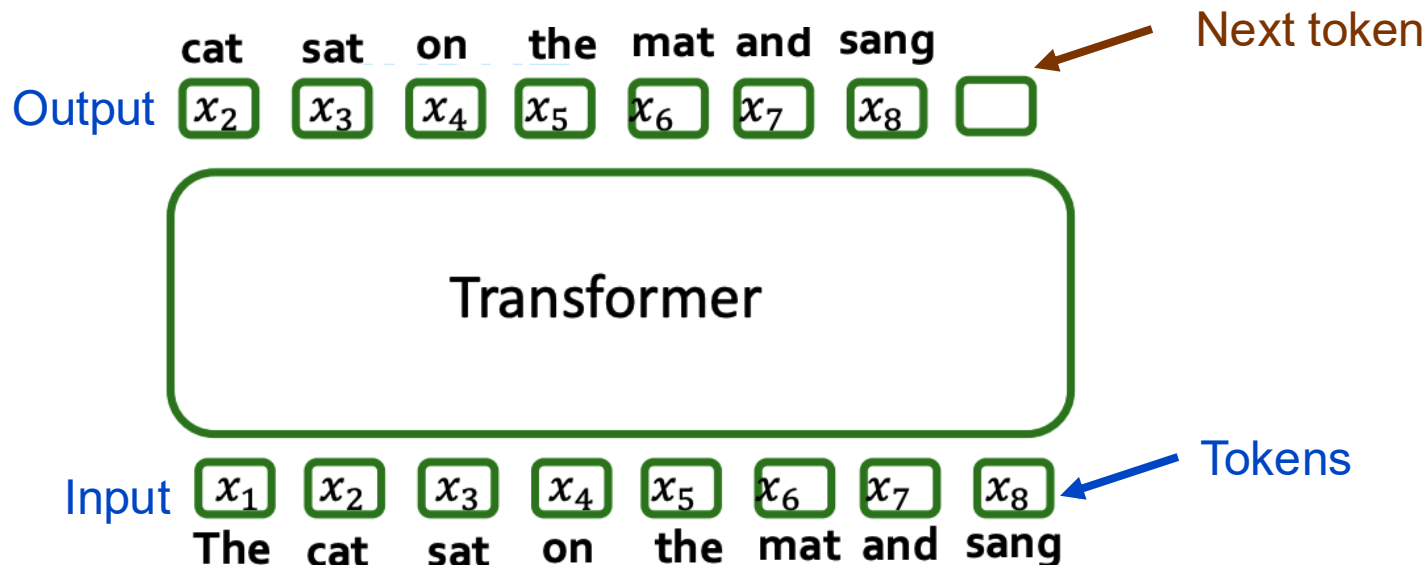
Note: There is a one-to-one correspondence between input and output vectors for tokens in the Transformer **encoder**. The transformer **decoder** is used to generate the next token.



Transformers and input tokens

- Transformers map sequences of vectors to sequences of vectors (**encoder**)
 - Each token describes a piece of the input, e.g. a word
- What is the output sequence?
 - Option 1:** next token (**decoder**) (this is what GPTs do)

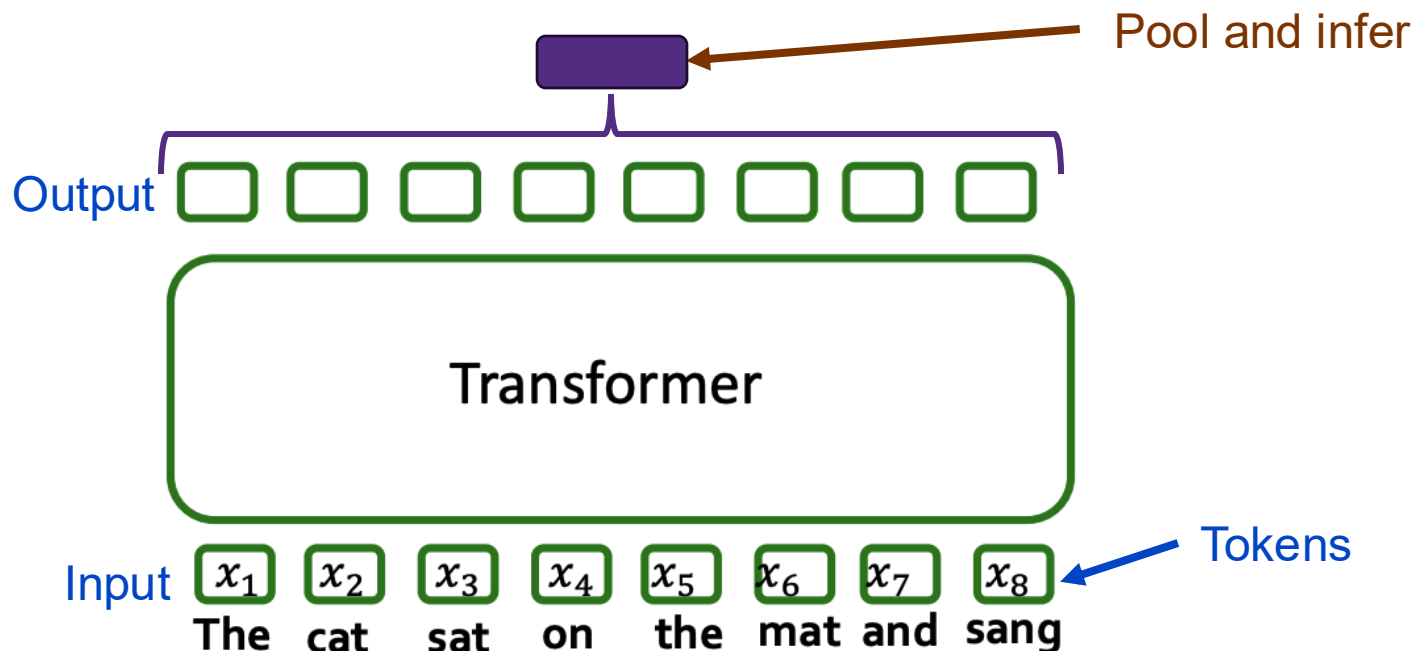
Note: There is a one-to-one correspondence between input and output vectors for tokens in the Transformer **encoder**. The transformer **decoder** is used to generate the next token.



Transformers and input tokens

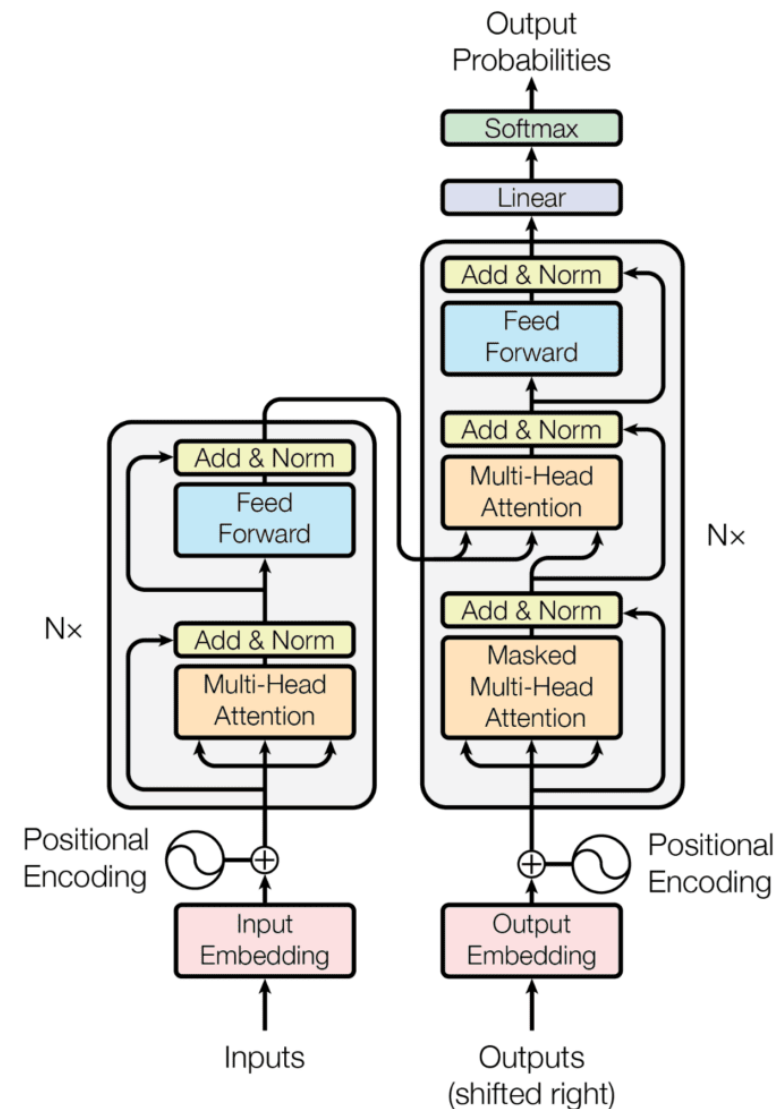
- Transformers map sequences of vectors to sequences of vectors
 - Each token describes a piece of the input, e.g. a word
- What is the output sequence?
 - Option 1: next token (**decoder**) (this is what GPTs do)
 - Option 2: pool and infer something (other modules) (e.g., document class)

Note: Here, the transformer **encoder** output is passed to other neural network models (e.g., MLP). The transformer **decoder** is not used.



Transformers architecture

- How are the input tokens processed?
- Several components
 - Positional encodings (more later)
 - **Multi-head self-attention**
 - Normalization
 - Feedforward networks
- What is self-attention and how does it work?





Transformer self-attention

- Multi-head attention blocks each individually perform self-attention
- Dynamically modify input representation to account for context

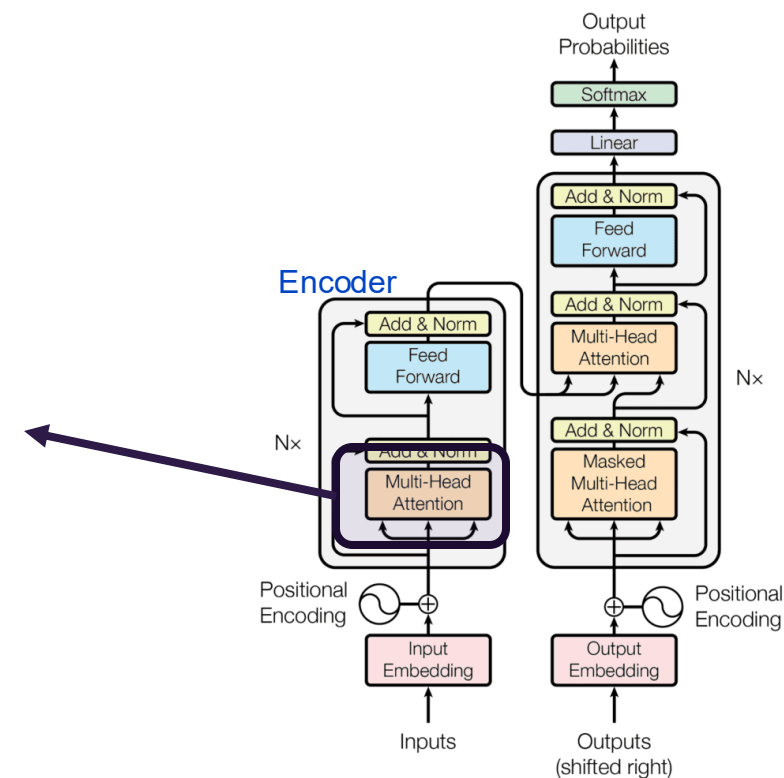
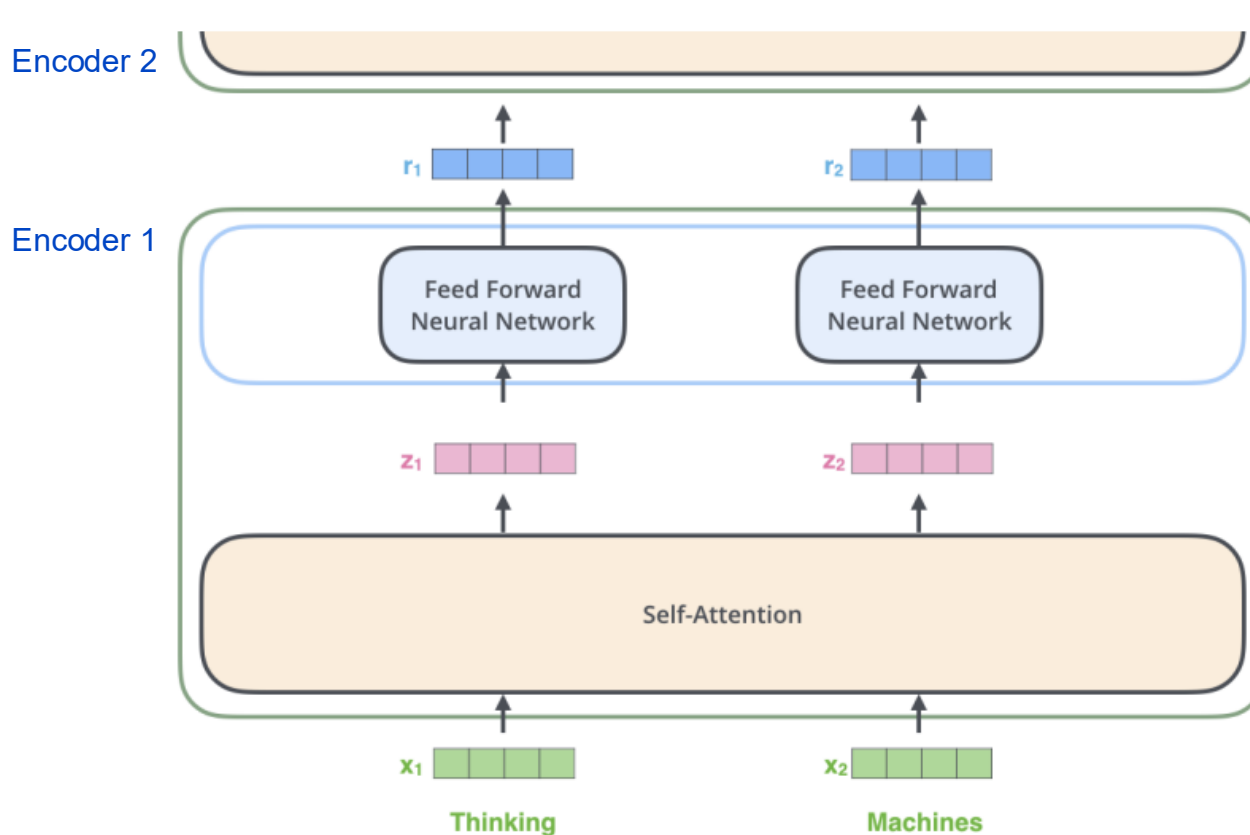
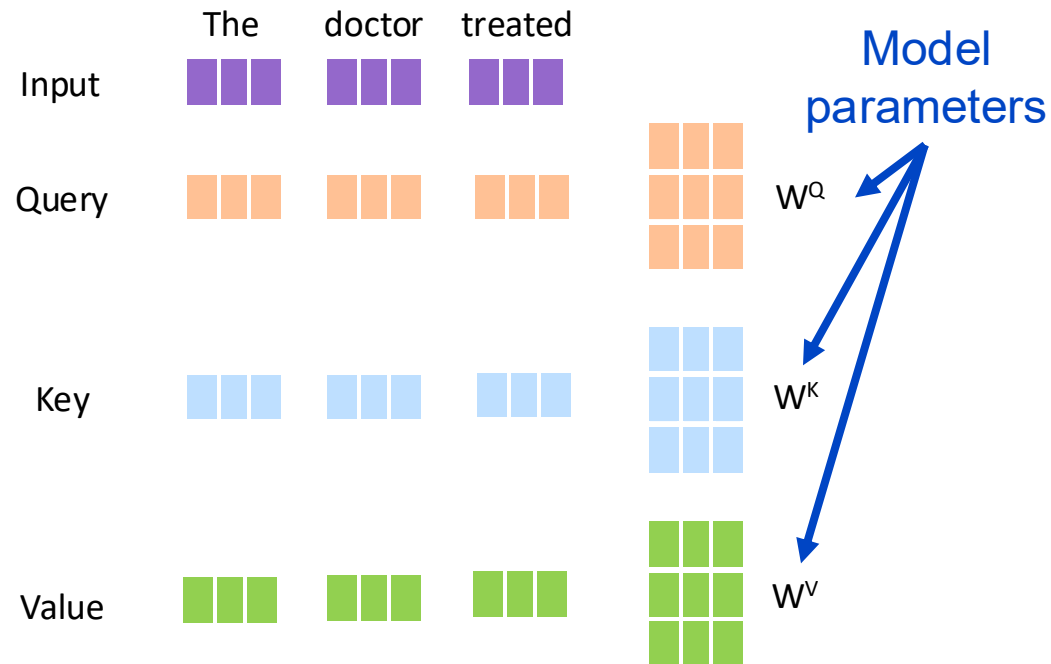


Image credit: Vaswani et, 2017

Transformer self-attention

- **Step 1:** compute *key*, *value*, and *query* for each input token

Step 1



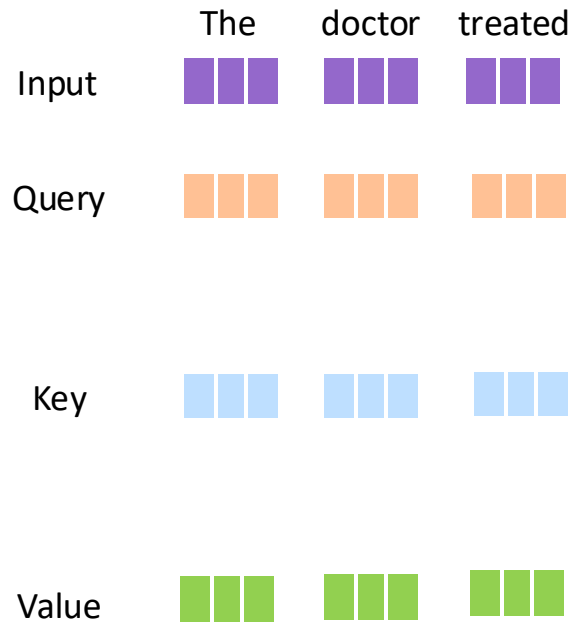
Note: Assume the input token embeddings $x_i \in \mathbb{R}^{d_{in}}$. Then for the first transformer encoder layer $W^Q, W^K, W^V \in \mathbb{R}^{d_{in} \times d_{out}}$. For each subsequent layer, $W^Q, W^K, W^V \in \mathbb{R}^{d_{out} \times d_{out}}$ (it is possible for these to use different d_{out})

A query, key, and value vector is generated for every input vector using the matrices W^Q , W^K , and W^V . These matrices are learned during model training.

Transformer self-attention

- **Step 1:** compute *key*, *value*, and *query* for each input token
- **Step 2:** For each token, compute scores for every other token and scale

Step 1



Step 2

Model
parameters

W^Q

W^K

W^V

Input

The
 x_1 [purple blocks]

doctor
 x_2 [purple blocks]

treated
 x_3 [purple blocks]

Query

q_1 [orange blocks]

q_2 [orange blocks]

q_3 [orange blocks]

Key

k_1 [blue blocks]

k_2 [blue blocks]

k_3 [blue blocks]

Value

v_1 [green blocks]

v_2 [green blocks]

v_3 [green blocks]

Score & Scale by
 $8\sqrt{d_k}$
Softmax

$q_1 \cdot k_1 = 14$

$q_1 \cdot k_2 = 12$

$q_1 \cdot k_3 = 4$

$c_{11} = 0.86$

$c_{12} = 0.12$

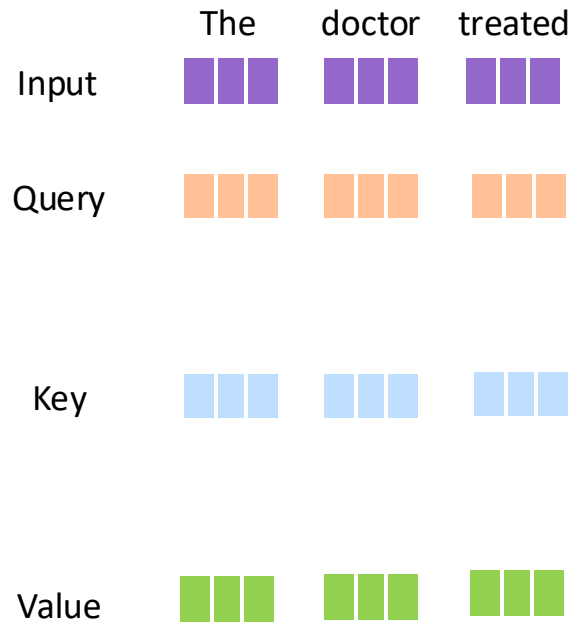
$c_{13} = 0.02$

The illustrated computations are for the computation of z_1 . The same process is repeated for every input x_i . For example, to compute z_3 , it is necessary to compute dot products q_3k_1 , q_3k_2 , q_3k_3 to obtain new softmax scores c_{31} , c_{32} , c_{33} and finally new scaled value vectors $c_{31}v_1$, $c_{32}v_2$ and $c_{33}v_3$.

Transformer self-attention

- **Step 1:** compute *key*, *value*, and *query* for each input token
- **Step 2:** For each token, compute scores for every other token and scale
- **Step 3:** Update embedding as a weighted sum of token value vectors

Step 1



Step 2

Model
parameters

W^Q

W^K

W^V

Input

The
 x_1 [purple blocks]

doctor
 x_2 [purple blocks]

treated
 x_3 [purple blocks]

Query

q_1 [orange blocks]

q_2 [orange blocks]

q_3 [orange blocks]

Key

k_1 [blue blocks]

k_2 [blue blocks]

k_3 [blue blocks]

Value

v_1 [green blocks]

v_2 [green blocks]

v_3 [green blocks]

Score & Scale by
 $8\sqrt{d_k}$

Softmax

$q_1 \cdot k_1 = 14$

$q_1 \cdot k_2 = 12$

$q_1 \cdot k_3 = 4$

$c_{11} = 0.86$

$c_{12} = 0.12$

$c_{13} = 0.02$

Softmax X
Value

$c_{11} v_1$ [yellow blocks]

$c_{12} v_2$ [yellow blocks]

$c_{13} v_3$ [yellow blocks]

Sum

$z_1 = c_{11} v_1 + c_{12} v_2 + c_{13} v_3$

Step 3

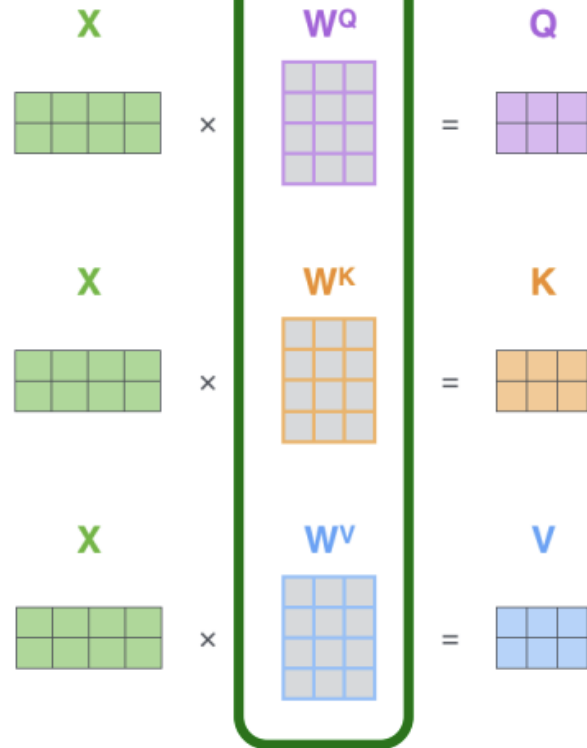
The illustrated computations are for the computation of z_1 . The same process is repeated for every input x_i . For example, to compute z_3 , it is necessary to compute dot products $q_3 k_1$, $q_3 k_2$, $q_3 k_3$ to obtain new softmax scores c_{31} , c_{32} , c_{33} and finally new scaled value vectors $c_{31} v_1$, $c_{32} v_2$ and $c_{33} v_3$.



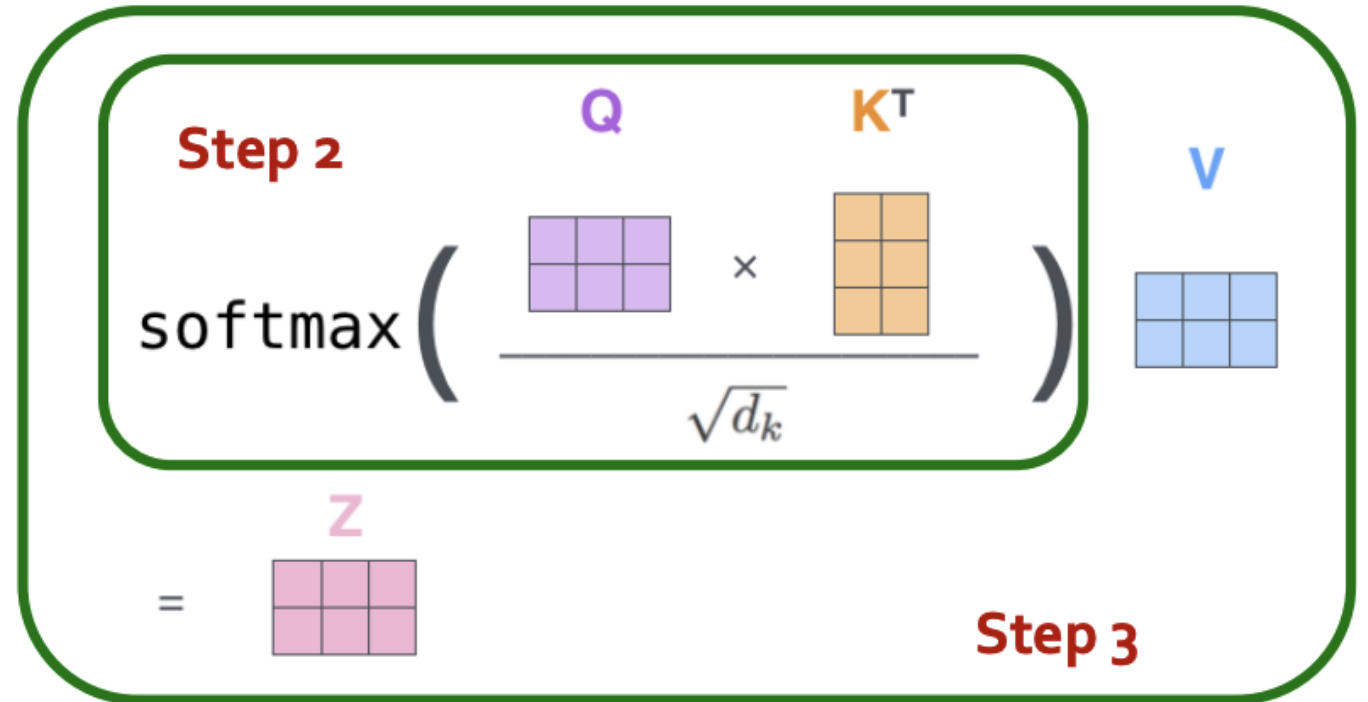
Transformer self-attention

Same calculation in matrix form

Step 1



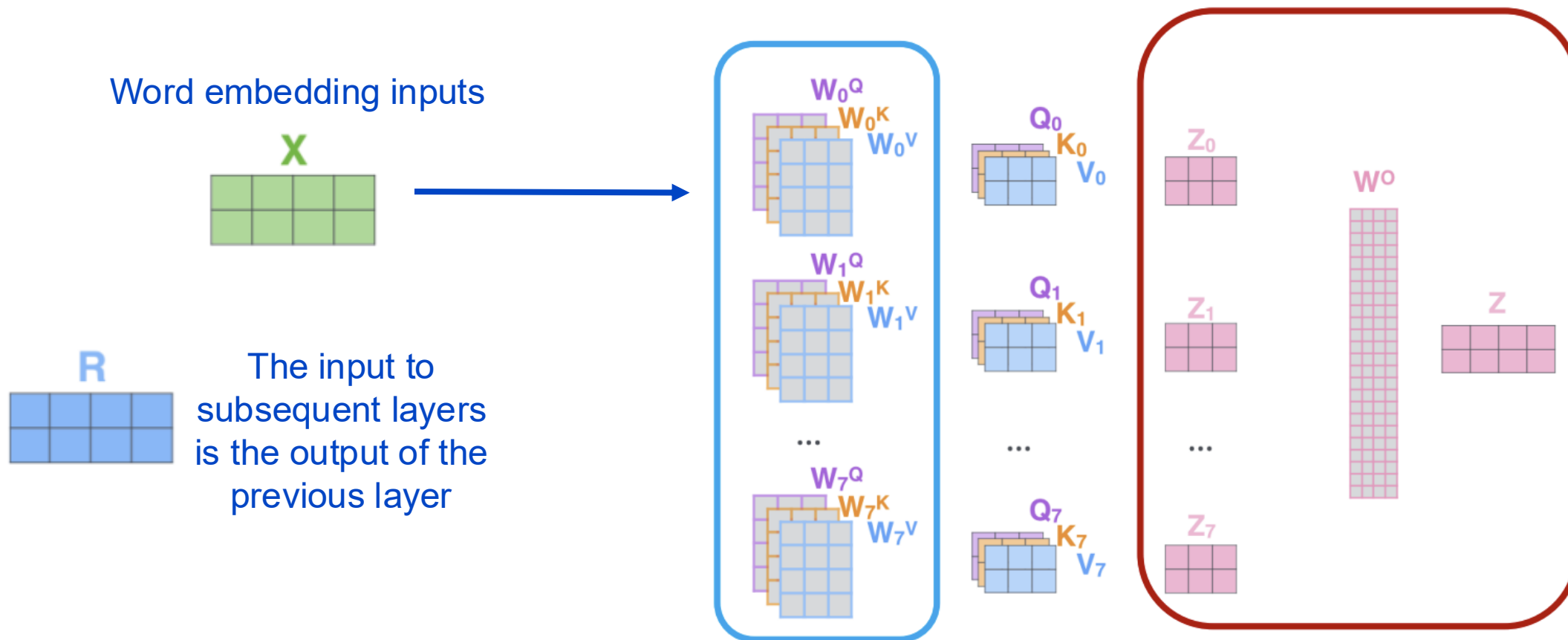
Model
parameters





Transformer multi-head self-attention

- Perform several self-attentions in parallel and combine results
- Different attention heads learn different relations among inputs
- **Each has its own learning parameters**



For more details on transformers for text, I highly recommend Jay Alammar's *The Illustrated Transformer*

<https://jalammar.github.io/illustrated-transformer/>

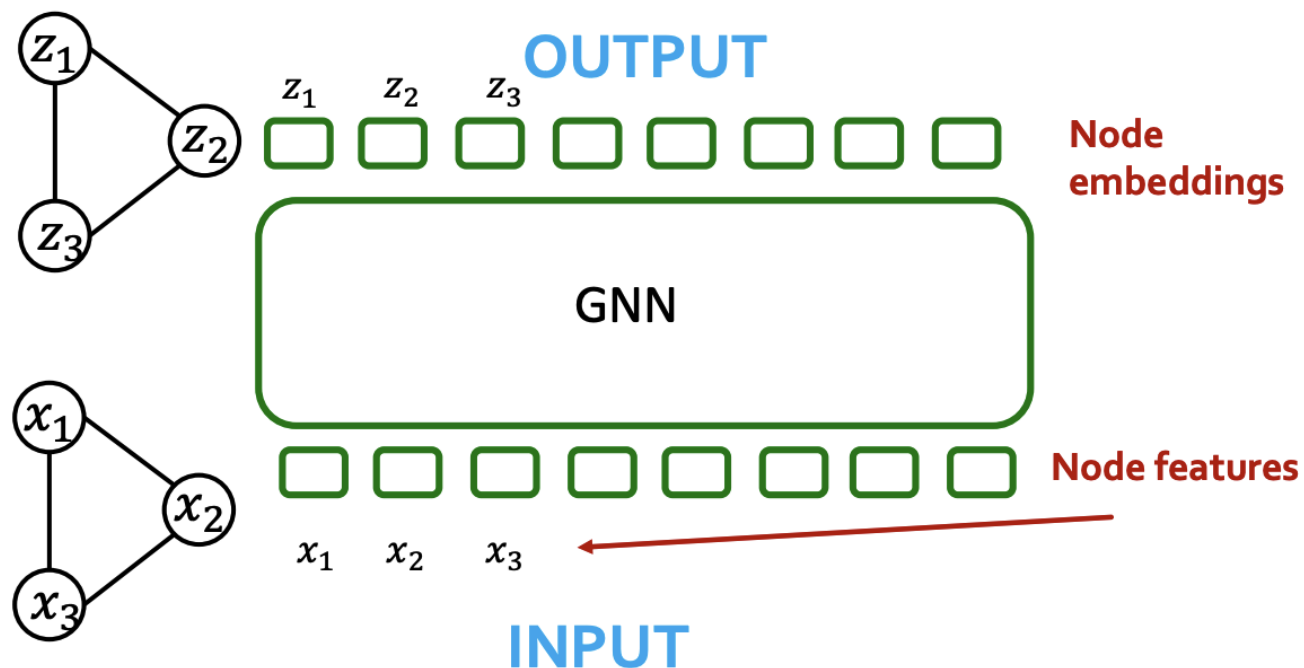


Transformers as a GNN



Comparing transformers and GNN

- Similarities
 - Both take a sequence of vectors and output embeddings
 - Both are permutation invariant and equivariant
- Differences
 - GNNs use **message passing**, Transformers use **self-attention**

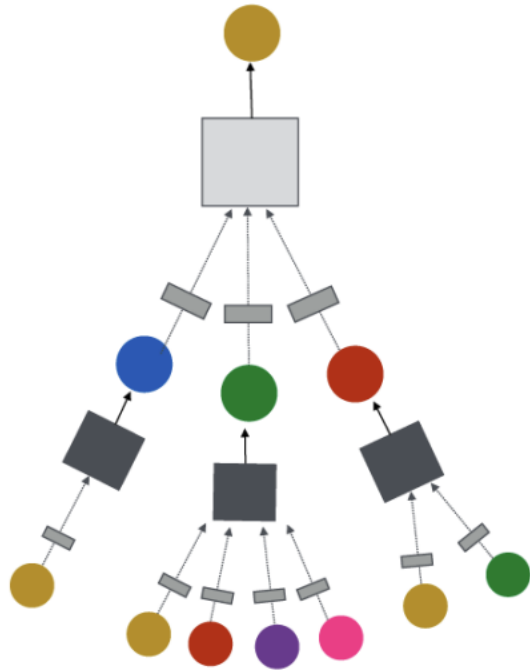




Comparing transformers and GNN

- Are self-attention and message passing really different?

Message Passing



?

Self-attention

$$\text{softmax}\left(\frac{\begin{matrix} Q \\ \text{2x2 grid} \end{matrix} \times \begin{matrix} K^T \\ \text{2x2 grid} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} V \\ \text{2x2 grid} \end{matrix}$$

$$= \begin{matrix} Z \\ \text{2x2 grid} \end{matrix}$$

$$\begin{matrix} X \\ \text{2x2 grid} \end{matrix} \times \begin{matrix} W^Q \\ \text{2x2 grid} \end{matrix} = \begin{matrix} Q \\ \text{2x2 grid} \end{matrix}$$

$$\begin{matrix} X \\ \text{2x2 grid} \end{matrix} \times \begin{matrix} W^K \\ \text{2x2 grid} \end{matrix} = \begin{matrix} K \\ \text{2x2 grid} \end{matrix}$$

$$\begin{matrix} X \\ \text{2x2 grid} \end{matrix} \times \begin{matrix} W^V \\ \text{2x2 grid} \end{matrix} = \begin{matrix} V \\ \text{2x2 grid} \end{matrix}$$



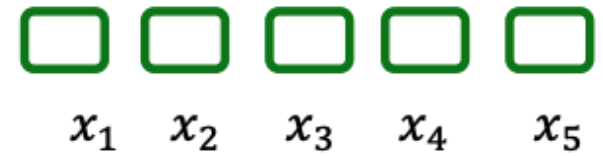
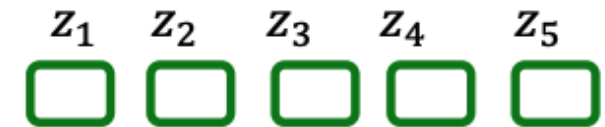
Self-attention can be cast as message passing

- The attention update: $Att(X) = softmax(QK^T)V$
where $Q = XW^Q, K = XW^K, V = XW^V$

Inputs stored row-wise

$$X = [-x_i -]$$

Output



Input tokens



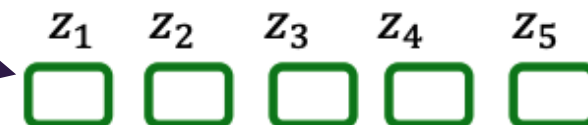
Self-attention can be cast as message passing

- The attention update: $Att(X) = softmax(QK^T)V$
where $Q = XW^Q$, $K = XW^K$, $V = XW^V$
- This formula gives the embeddings for **all tokens** simultaneously
- Let's simplify to consider just one token, x_1

Inputs stored row-wise

$$X = [-x_i -]$$

Output



Input tokens

Self-attention can be cast as message passing

- The attention update: $Att(X) = softmax(QK^T)V$ where $Q = XW^Q, K = XW^K, V = XW^V$
- This formula gives the embeddings for **all tokens** simultaneously
- Let's simplify to consider just one token, x_1

$$z_1 = \sum_{j=1}^5 softmax_j(q_1^T k_j) v_j$$

Updated embedding for x_1
 $z_1 \in \mathbb{R}^{1 \times d_{out}}$

Query vector
 $q_1 = x_1 W^Q$

Key vector
 $k_j = x_j W^K$

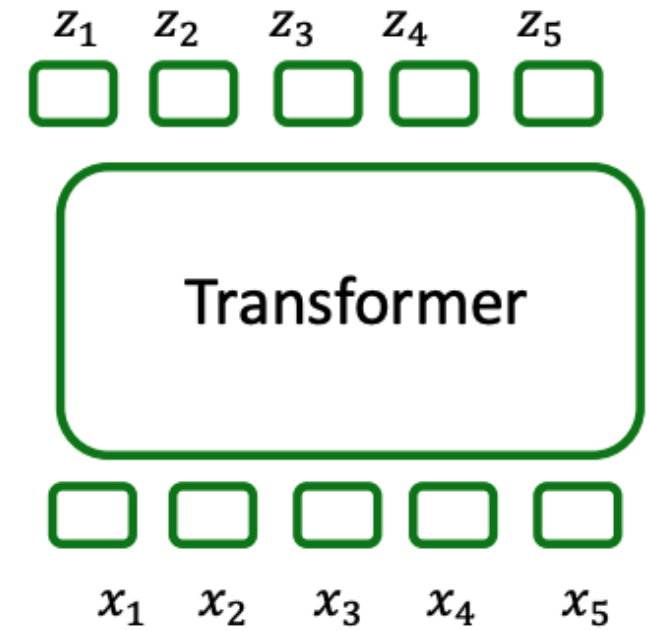
Value vector
 $v_j = x_j W^V$

How should we interpret this?

Inputs stored row-wise

$$X = [-x_i -]$$

Output



Input tokens



Self-attention can be cast as message passing

- The attention update: $Att(X) = softmax(QK^T)V$

where $Q = XW^Q$, $K = XW^K$, $V = XW^V$

Inputs stored row-wise

$$X = [-x_i -]$$

- This formula gives the embeddings for **all tokens** simultaneously
- Let's simplify to consider just one token, x_1

$$z_1 = \sum_{j=1}^5 softmax_j(q_1^T k_j) v_j$$

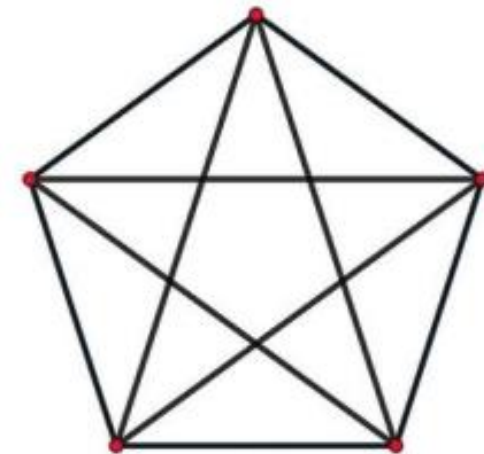
- Steps for computing the embedding, z_1
 - The **messages** from other tokens are x_j
 - The messages are **aggregated** by $Agg_j(x_1 W^Q, x_j W^K, x_j W^V) = \sum_{j=1}^5 softmax_j(q_1^T k_j) v_j$
 - The update is just the identity, i.e., $z_1 = 1 * Agg_j$



Self-attention can be cast as message passing

- Self-attention can be viewed as message aggregation
- What is the graph?
 - Nodes = tokens.
 - Since each token depends on every other token for updating, the graph is fully connected
- If summing only over $j \in N(i) \Rightarrow$ GAT
- Steps for computing the embedding, z_1
 1. The **messages** from other tokens are x_j
 2. The messages are **aggregated** by

$$Agg_j(x_1 W^Q, x_j W^K, x_j W^V) = \sum_{j=1}^5 softmax_j(q_1^T k_j) v_j$$
 3. The update is just the identity, i.e., $z_1 = 1 * Agg_j$

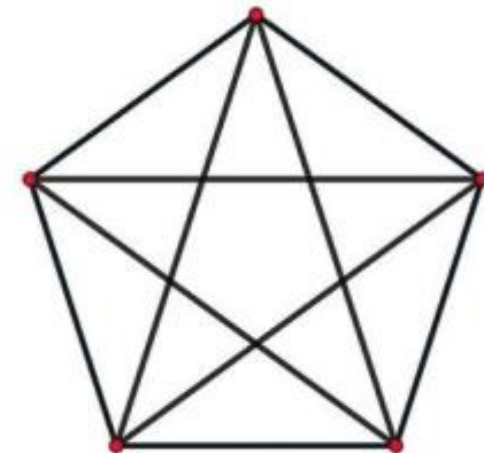




Self-attention can be cast as message passing

- **Takeaway 1:** Self-attention is a special case of message passing
- **Takeaway 2:** It is message passing on a **fully connected** graph
- **Takeaway 3:** Given a graph, G , if you constrain the self-attention to only be over j adjacent tokens of $i \Rightarrow$ GAT
- Steps for computing the embedding, z_1
 1. The **messages** from other tokens are x_j
 2. The messages are **aggregated** by

$$Agg_j(x_1 W^Q, x_j W^K, x_j W^V) = \sum_{j=1}^5 softmax_j(q_1^T k_j) v_j$$
 3. The update is just the identity, i.e., $z_1 = 1 * Agg_j$





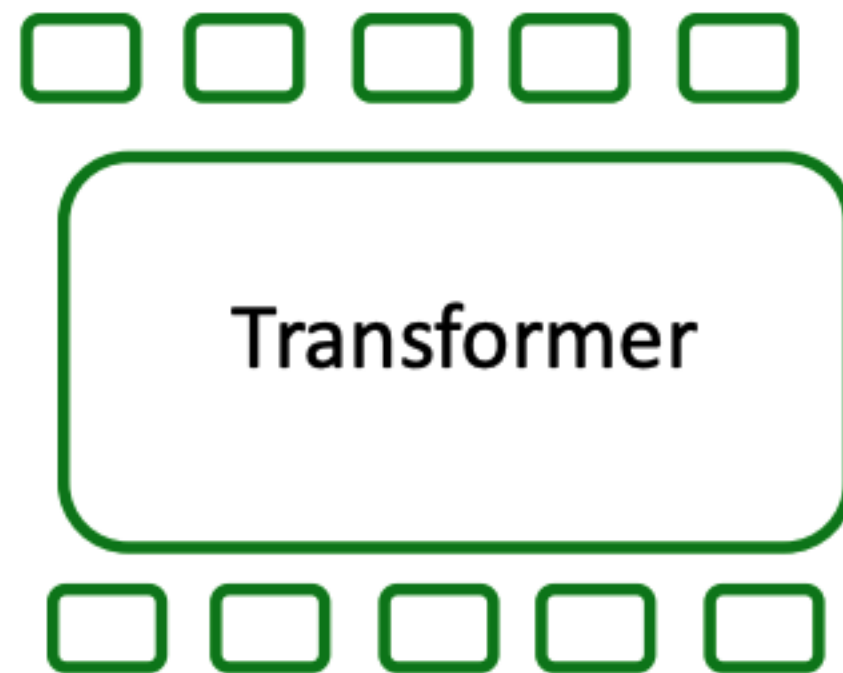
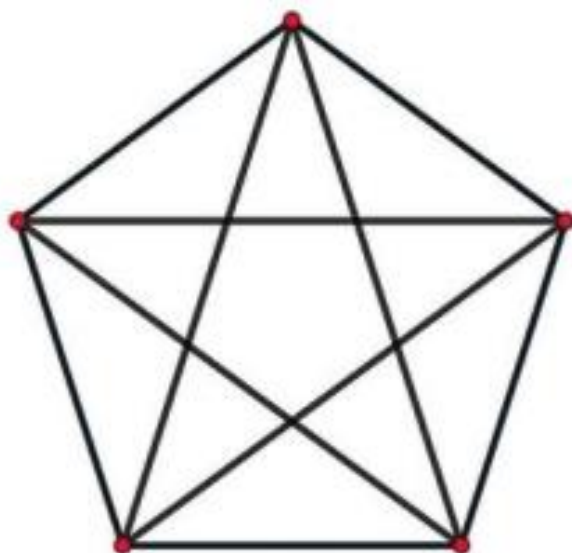
Transformers on graphs





Processing graphs with transformers

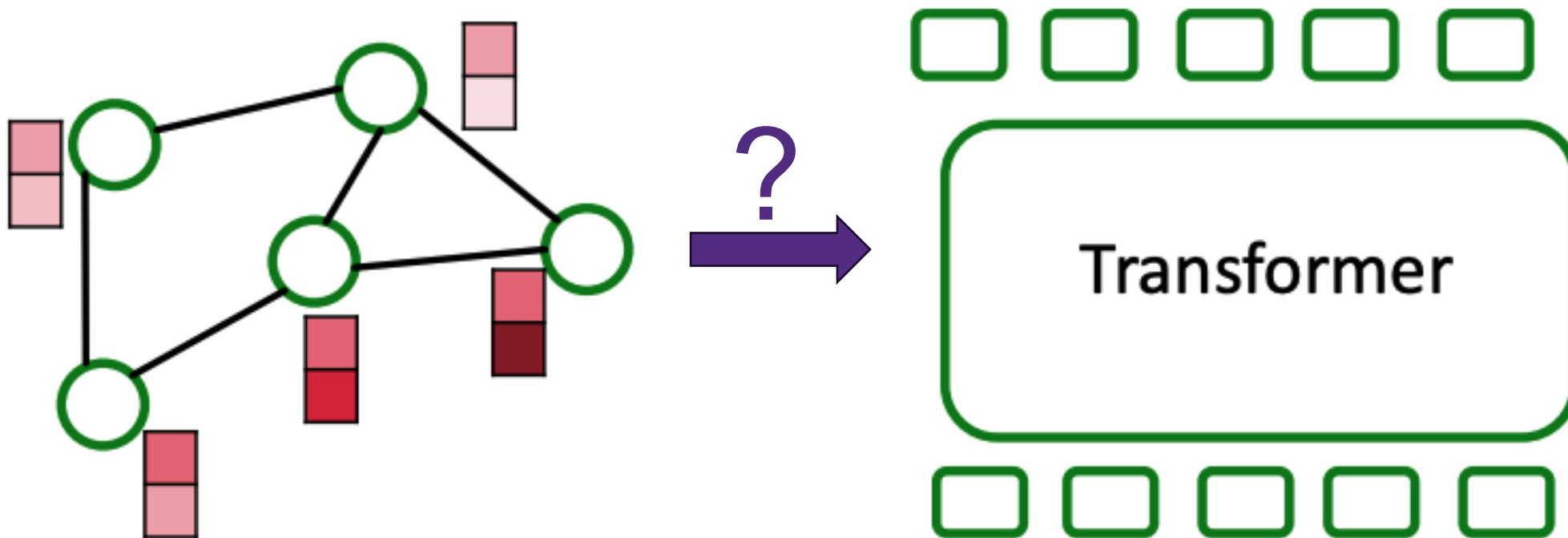
We've seen how to go from a transformer on tokens to a graph





Processing graphs with transformers

How should we input an arbitrary graph into a transformer?

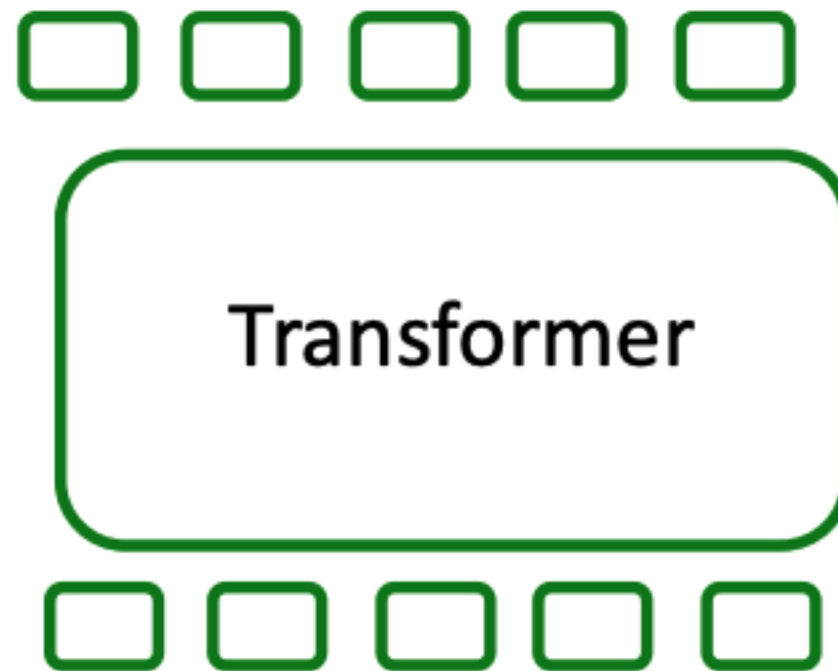




Processing graphs with transformers

How should we input an arbitrary graph into a transformer?

- Let's review the key components of the transformer
 - Token inputs
 - Self-attention
 - **Is there anything else?**



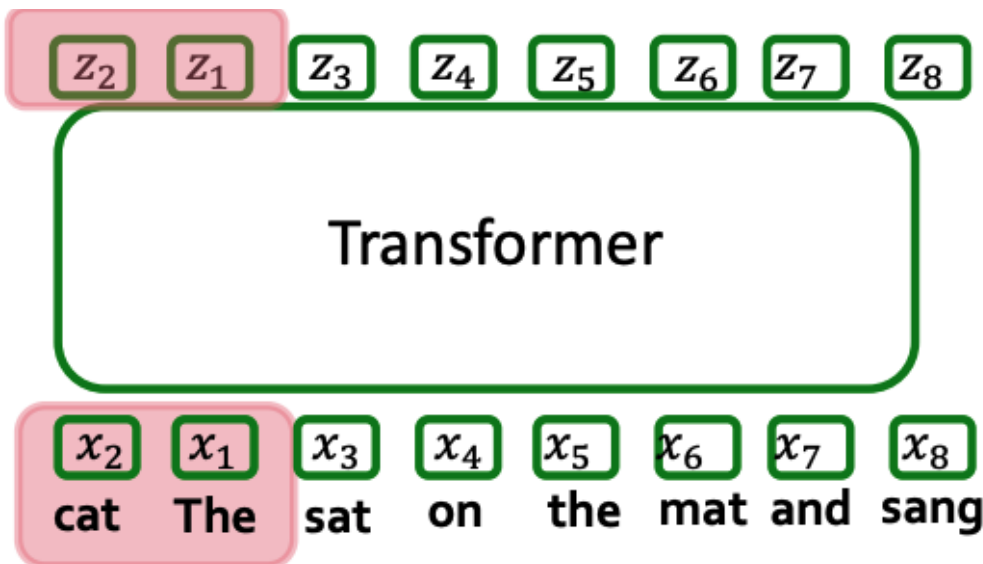
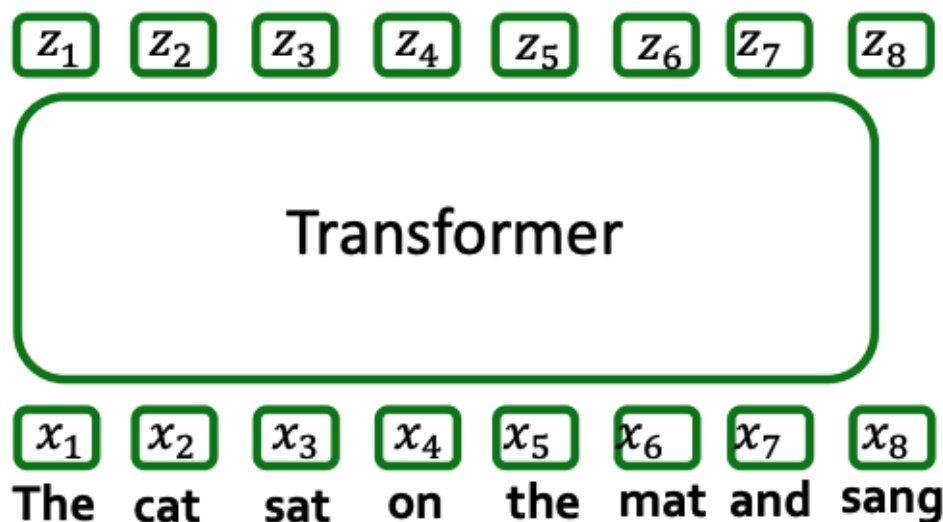


What about token ordering?

- Token order does not affect self-attention output

$$z_1 = \sum_{j=1}^5 \text{softmax}_j(q_1^T k_j) v_j$$

Outputs change position, but not values

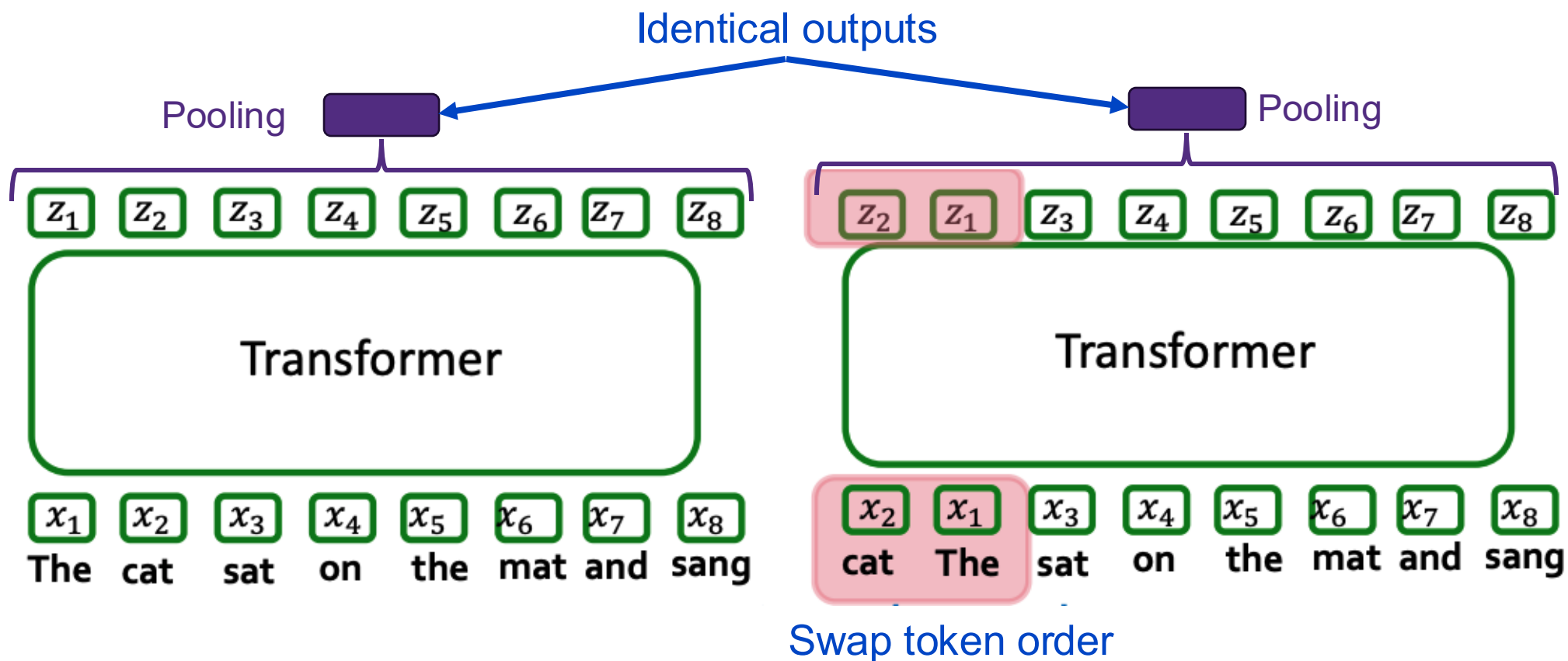


Swap token order



What about token ordering?

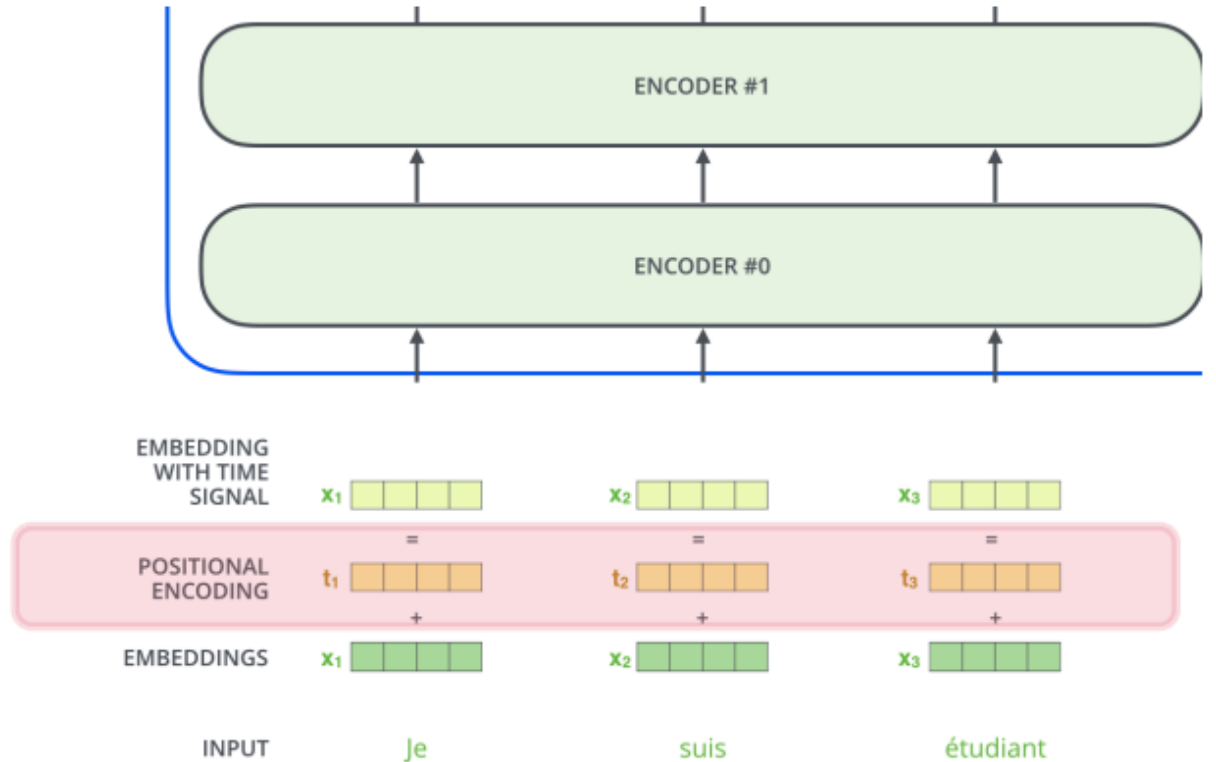
- Same predictions independent of word order
- Equivalent to a bag of words model
- Can we fix this?





Positional encodings

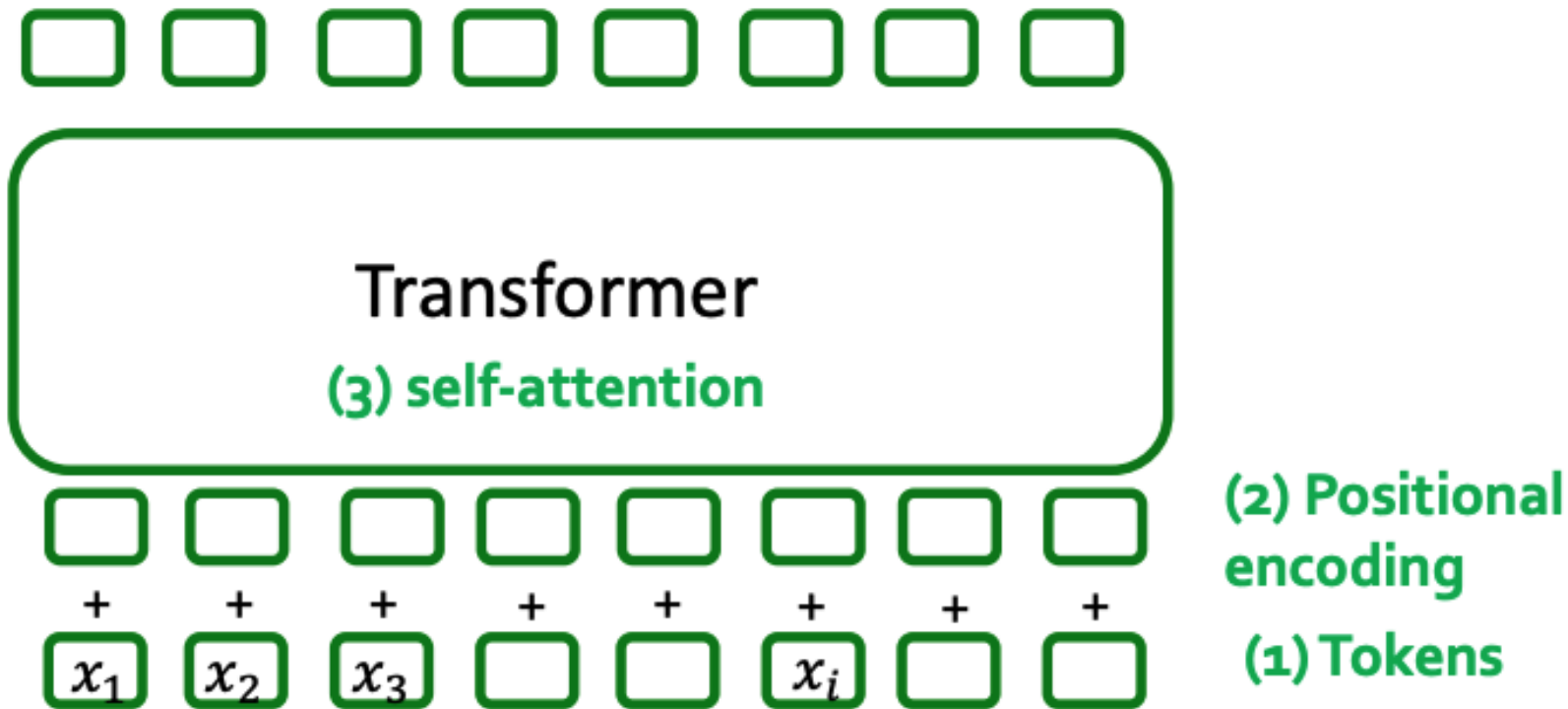
- Add positional features to input tokens to encode word order information
 - Je = word 1
 - suis = word 2
 - etc
- For NLP, the positional encoding vectors are learnable parameters





Transformer components

How should we use each of these for graphs?

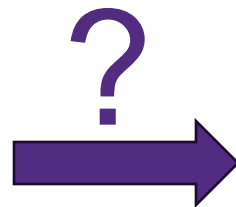




Processing graphs with transformers

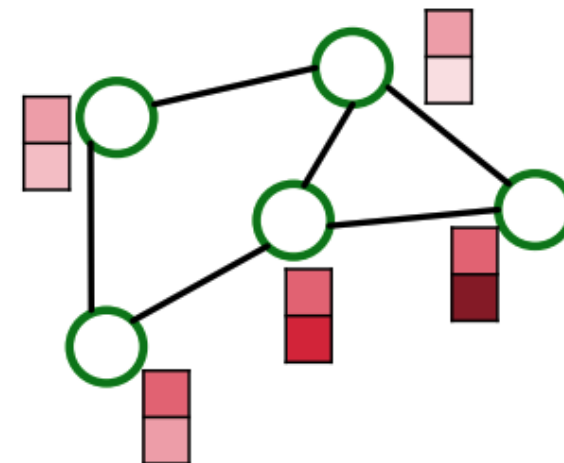
- We need our transformer to take the following inputs:

1. Node features
2. Adjacency (edge) information
3. Edge features



- Transformer components

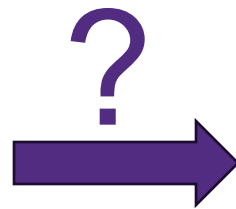
- A. Token inputs
- B. Positional encoding
- C. Self-attention



Processing graphs with transformers

- We need our transformer to take the following inputs:

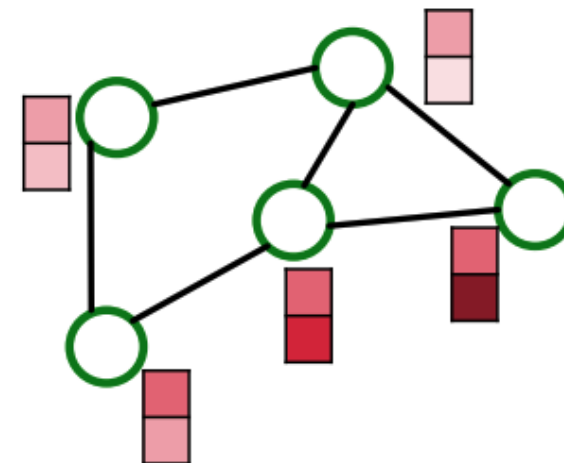
1. Node features
2. Adjacency (edge) information
3. Edge features



- Transformer components

- A. Token inputs
- B. Positional encoding
- C. Self-attention

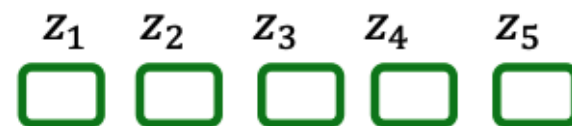
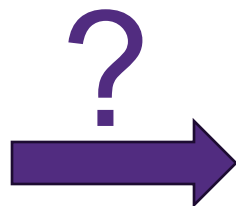
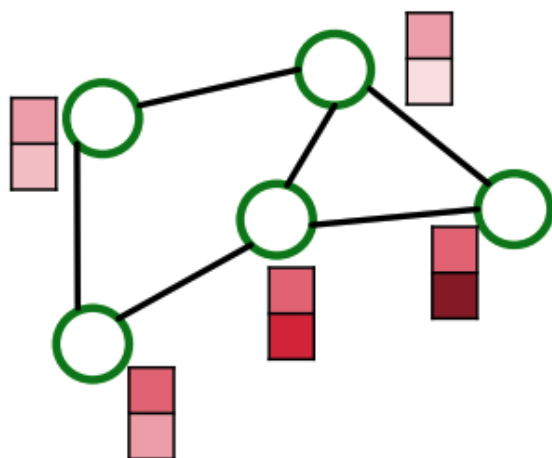
- Different ways to do this mapping
- We'll consider
 - $1 \rightarrow A$
 - $2 \rightarrow B$
 - $3 \rightarrow C$





Nodes as tokens

- Use node features vectors as input tokens
- Matches the idea of “attention as message passing” for a **fully connected** graph

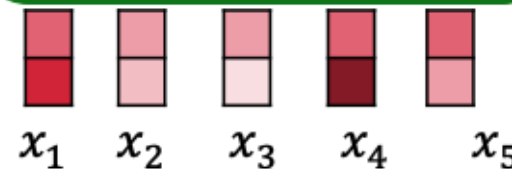
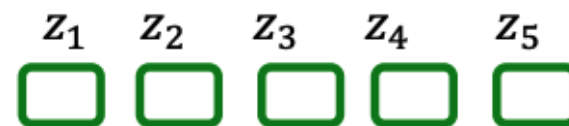
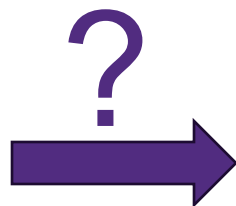
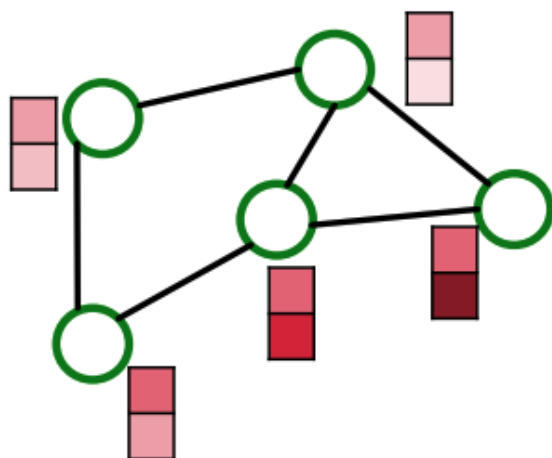


Input tokens = Node features



Nodes as tokens

- Use node features vectors as input tokens
- Matches the idea of “attention as message passing” for a **fully connected** graph
- How to include **adjacency** information for a graph that is **not fully connected**?

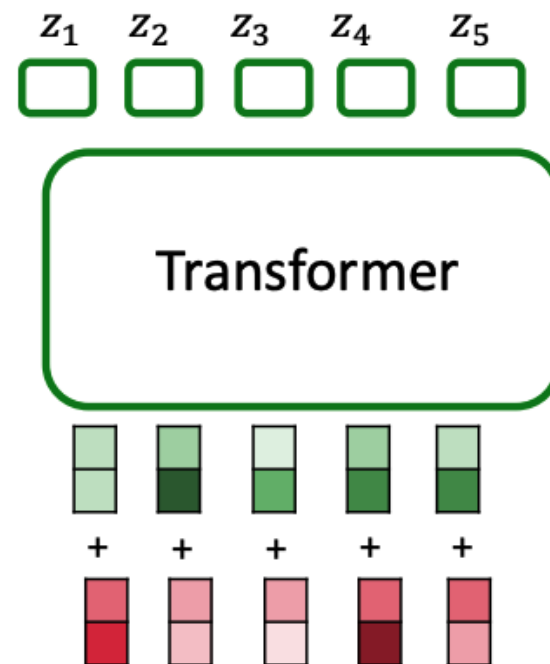
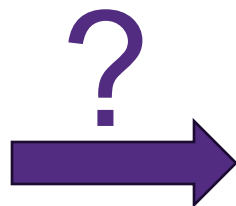
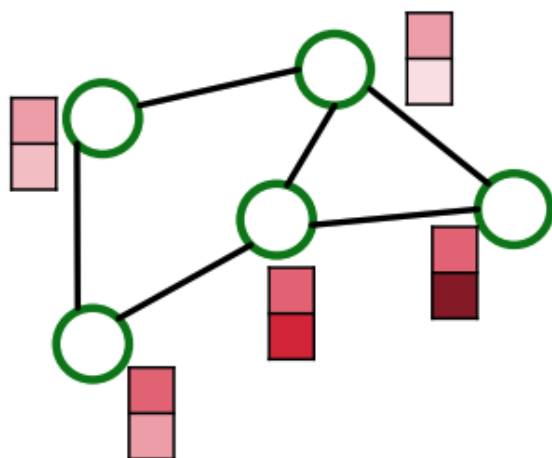


Input tokens = Node features



Positional encodings for adjacency?

- Encode the adjacency information in the positional encodings
- Indicates *where* a node is in the graph
- How do we design the encoding?

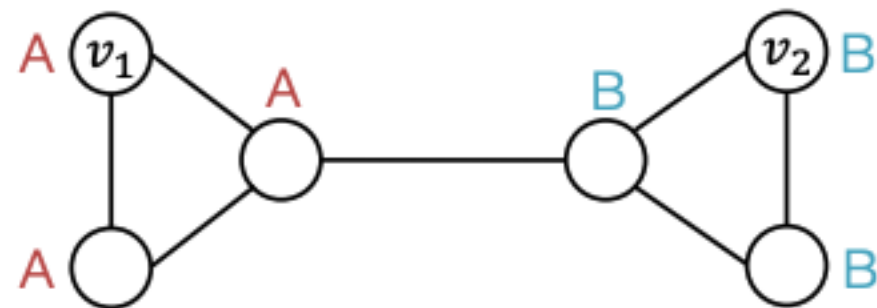
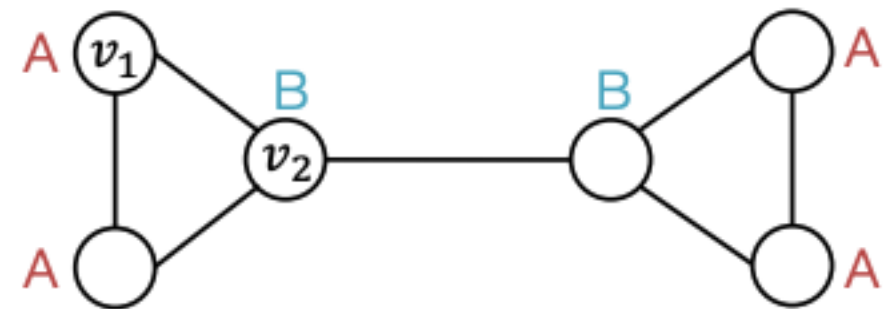


Positional encoding?

Input tokens = Node features

Two types of tasks on graphs

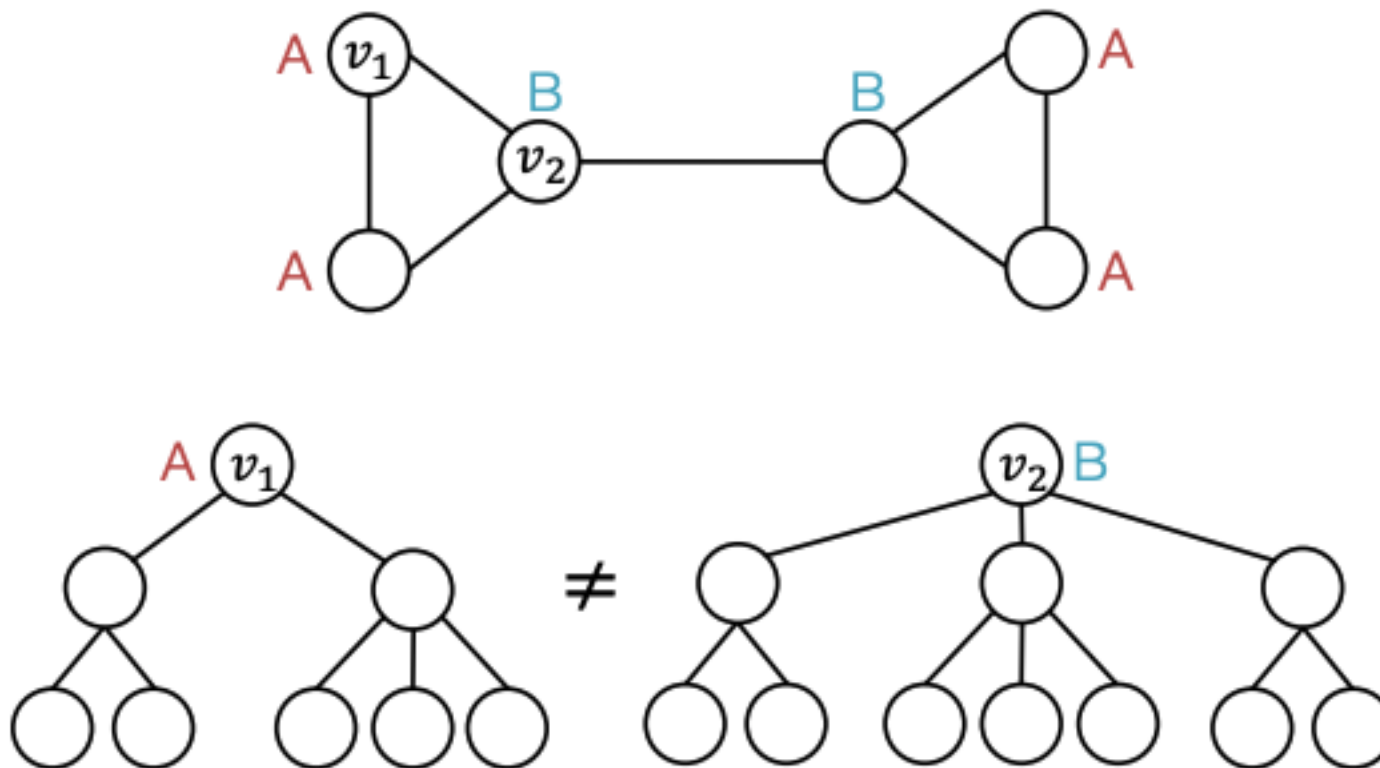
- Structure-aware tasks – nodes are labeled by their **structural roles** in the graph
- Position-aware tasks are labeled by their **positions** in the graph





Structure-aware tasks

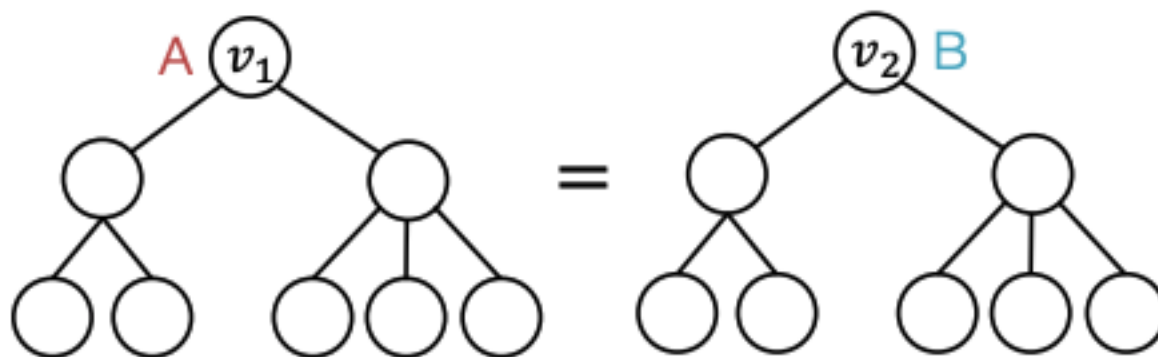
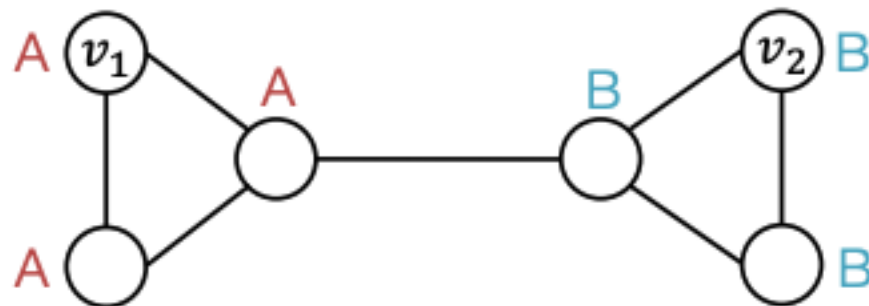
We've seen that GNNs work well for differentiated nodes with different structural roles because they have different computation graphs





Position-aware tasks

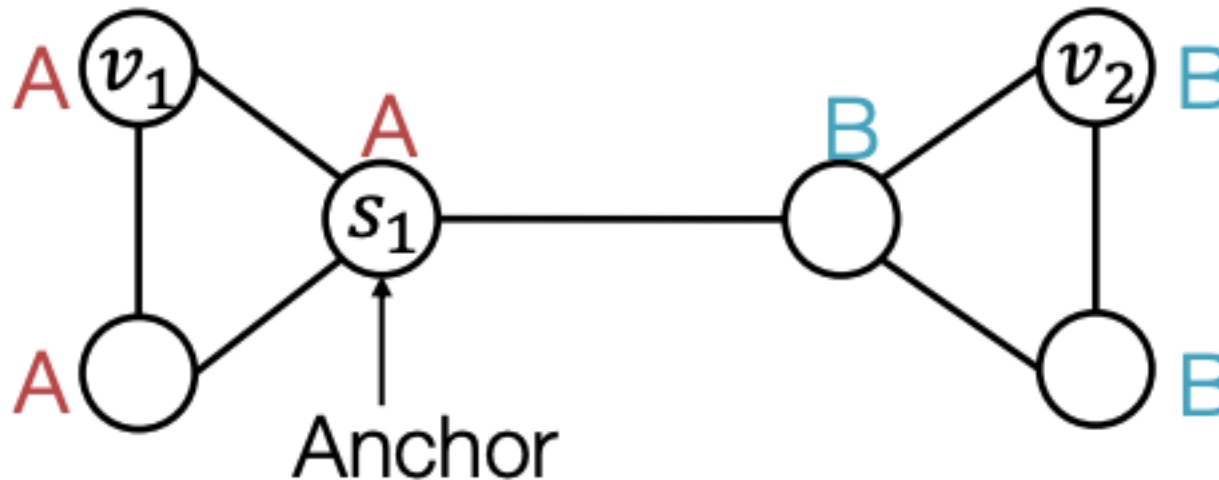
- Ignoring node features, GNNs will fail on position-aware tasks if the computation graphs are the same due to structural symmetry
- Can we address this?





Anchors for position aware graph tasks

- Randomly pick a node s_1 as an anchor
- Represent nodes v_1 and v_2 by their edge distance to the anchor
- Anchor node serves as a coordinate axis



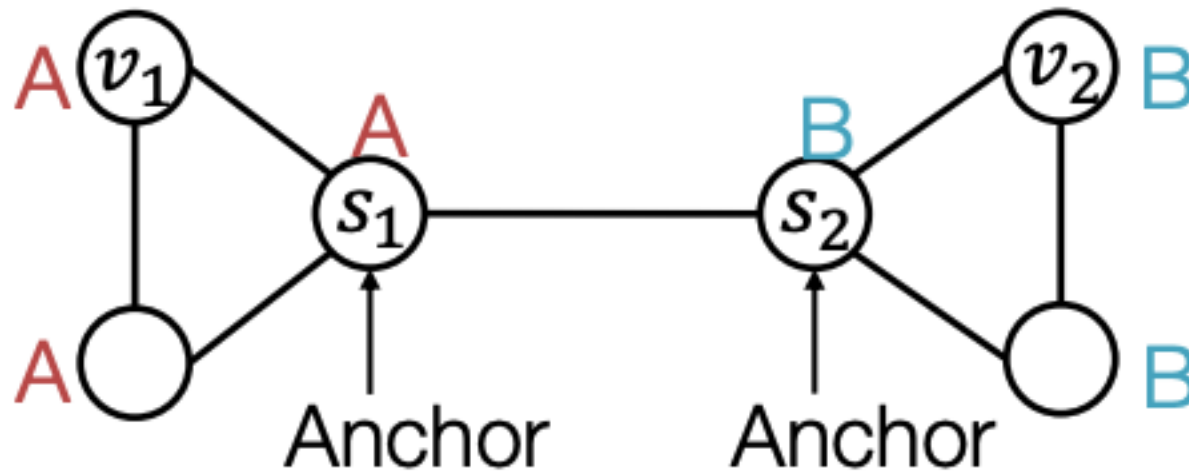
Edge Distances

	s_1
v_1	1
v_2	2



Anchors for position aware graph tasks

- Randomly pick more anchor nodes
- More anchors can better characterize node position relative to different regions of the graph
- Multiple anchors correspond to multiple coordinate axes

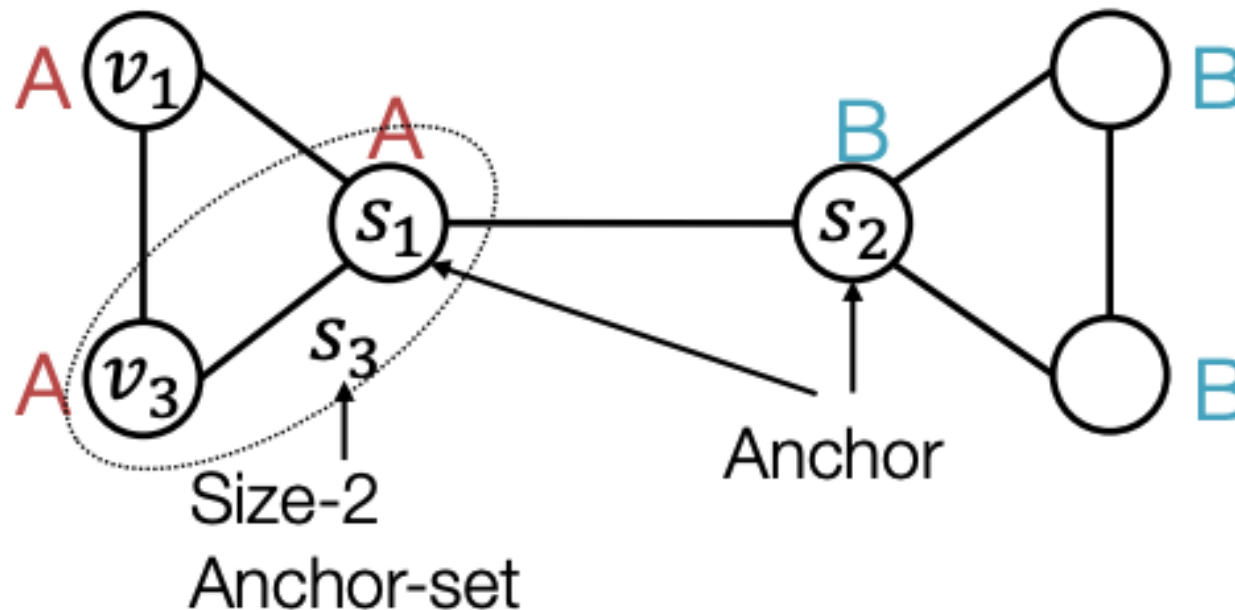


Edge Distances

	s_1	s_2
v_1	1	2
v_2	2	1

Anchors for position aware graph tasks

- Generalize to an **anchor-set** – i.e. a set of nodes used as anchors
- Define the distance from a node v to an anchor-set as the minimum of the edge distances to the nodes in the set



Edge Distances

	s_1	s_2	s_3
v_1	1	2	1
v_3	1	2	0

Anchors s_1 , s_2 cannot differentiate nodes v_1 , v_2 but anchor s_3 can



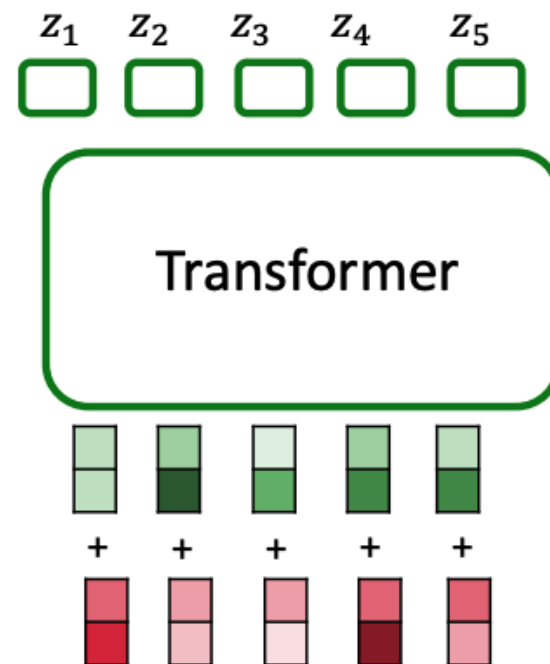
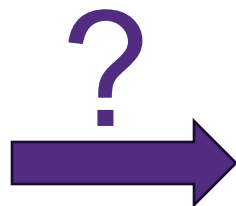
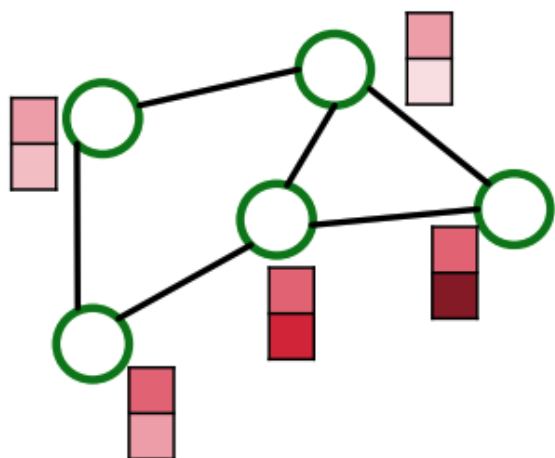
How is anchor-set information used in GNNs?

- Use position encoding as an augmented node feature (works well in practice)
 - Issue: this position encoding is not permutation invariant
- A rigorous solution is detailed in Position-aware GNNs by You et al.



Positional encodings for adjacency?

- Encode the adjacency information in the positional encodings
- Indicates *where* a node is in the graph
- How do we design the encoding?



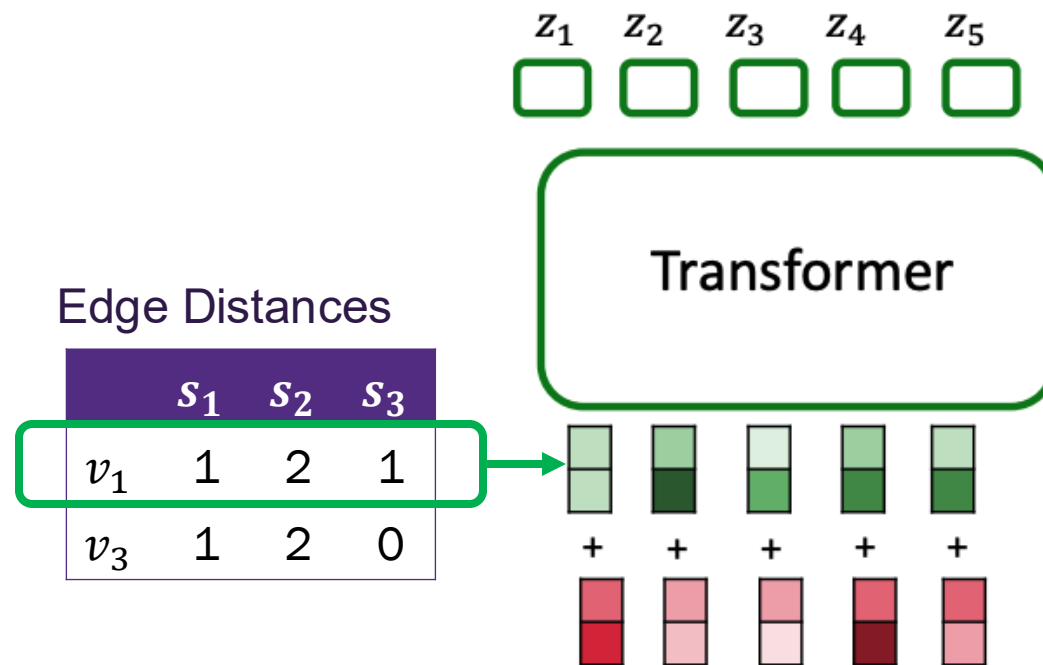
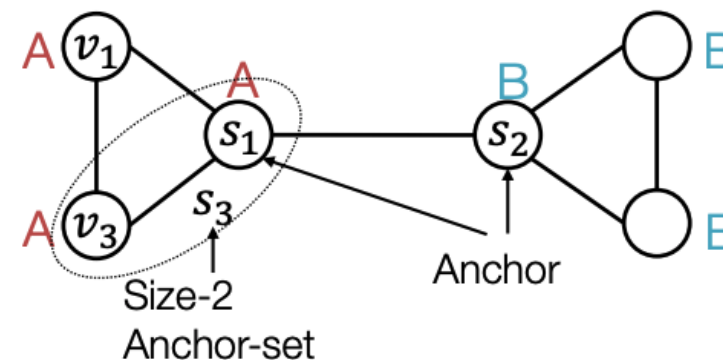
Positional encoding?

Input tokens = Node features



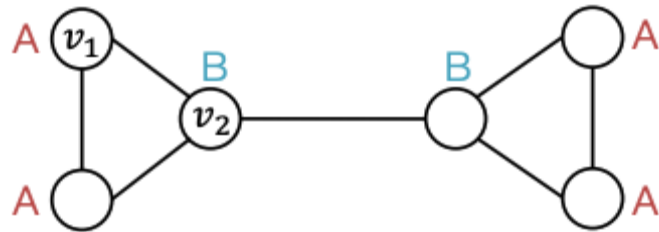
Relative distances for transformer positional encoding

- Works well in many cases
- Especially good for tasks that require counting cycles

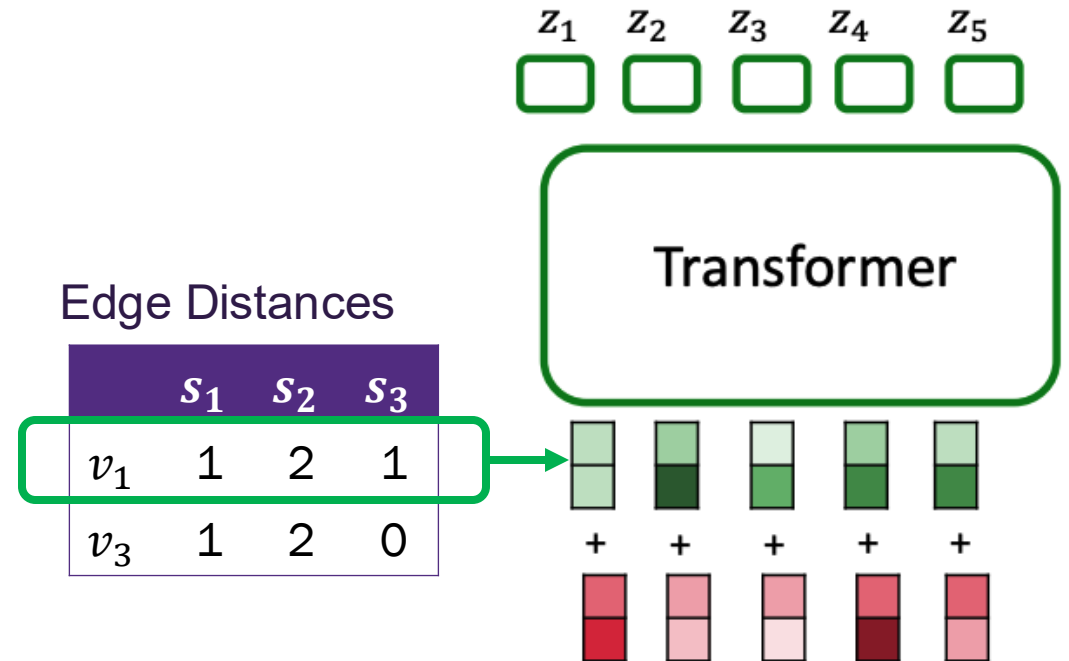
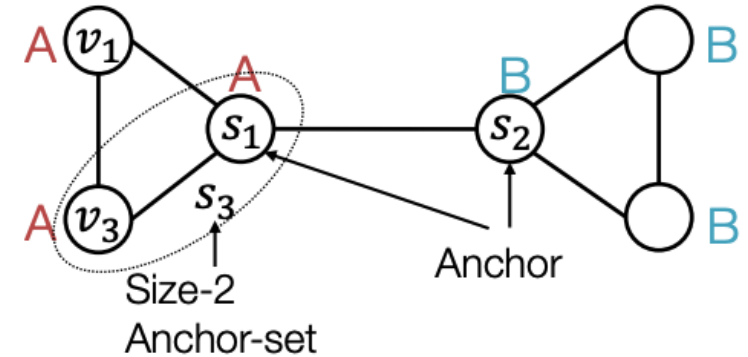


Relative distances for transformer positional encoding

- Works well in many cases
- Especially good for tasks that require counting cycles
- But not well suited for structure-aware tasks



- What else can we do?





Graph Laplacian

- The graph Laplacian, $\mathbf{L}$, of a **simple undirected network** is an $n \times n$ matrix with elements

$$l_{ij} = k_i \delta_{ij} - a_{ij} = \begin{cases} k_i & \text{if } i = j \\ -1 & \text{if } i \neq j \text{ and there is an edge from } i \text{ to } j \\ 0 & \text{otherwise} \end{cases}$$

where k_i is the degree of node i and δ_{ij} is the Kronecker delta (1 if $i = j$ and 0 otherwise)

- Matrix form $\mathbf{L} = \mathbf{D} - \mathbf{A}$ where

$$\mathbf{D} = \begin{pmatrix} k_1 & 0 & \cdots \\ 0 & k_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

- For **weighted networks**, substitute the weighted adjacency matrix and $k_i = \sum_j a_{ij}$
- No extensions for networks with self-edges or directed networks



Laplacian Eigenvector positional encodings

- Laplacian matrix, $\mathbf{L}$, captures graph structure
- Its Eigenvectors encode this structure
- Let's use these as positional encoding for transformers

Refresher

$$\mathbf{L}\mathbf{x} = \lambda\mathbf{x}$$

$\mathbf{L}$: $n \times n$ matrix, where n is the number of nodes

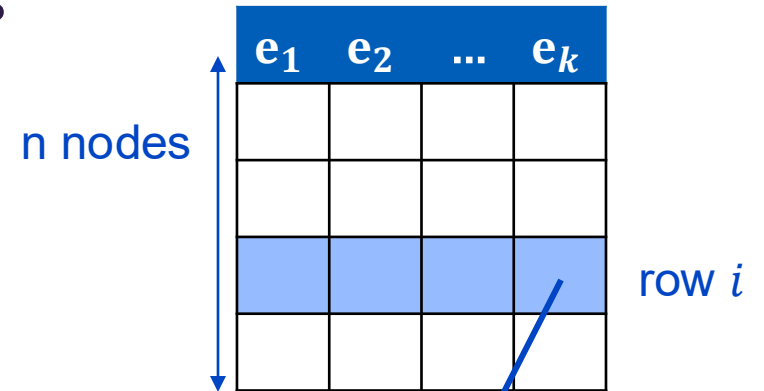
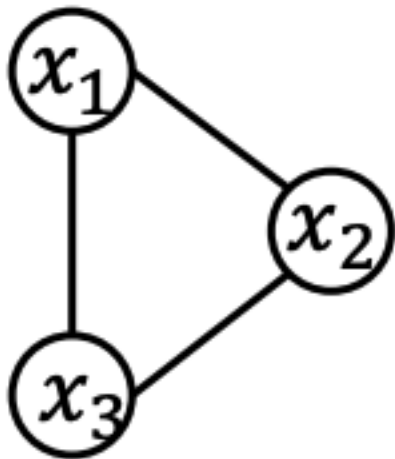
$\mathbf{x}$: n dimensional vector

λ : scalar Eigenvalue



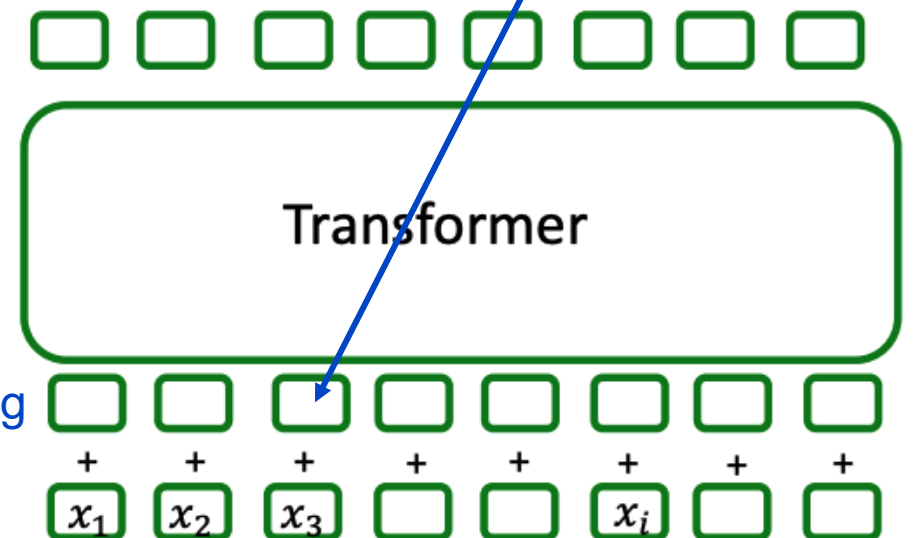
Laplacian Eigenvector positional encodings

- Creating positional encodings from Eigenvectors
 - Compute k Eigenvectors, $\mathbf{e}_k$
 - Stack left to right to form a matrix
 - i^{th} row is positional encoding for node v_i



Positional encoding

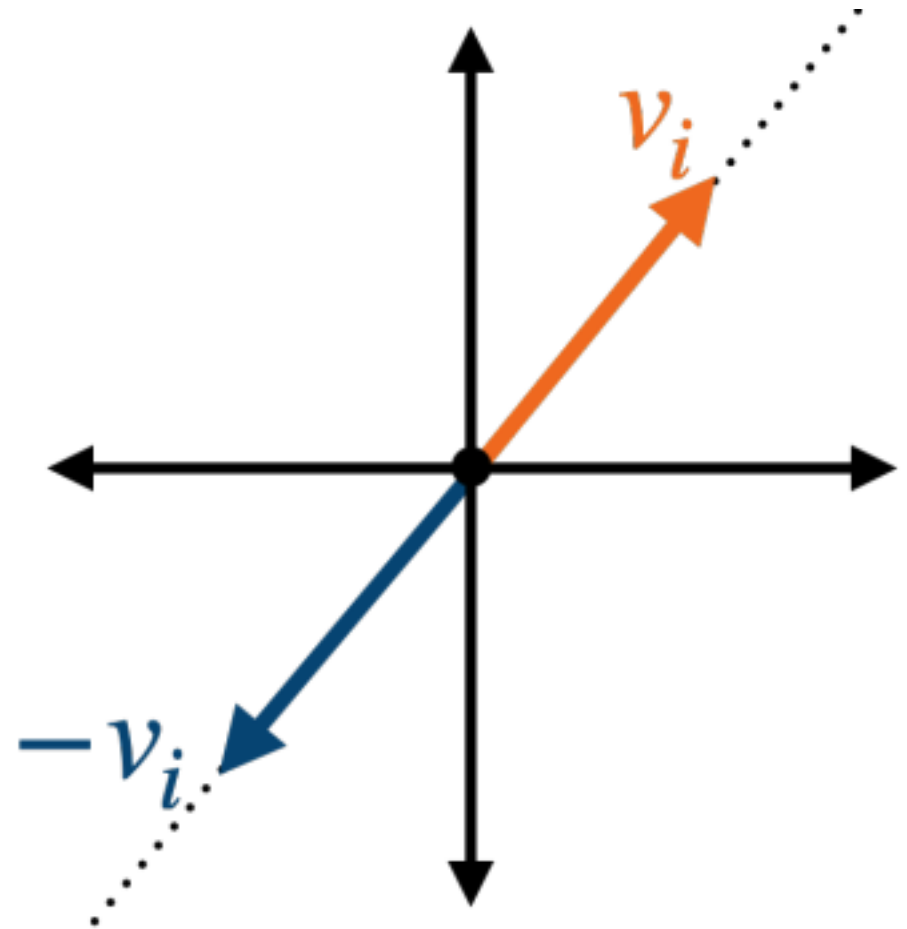
Node features





Eigenvector sign ambiguity

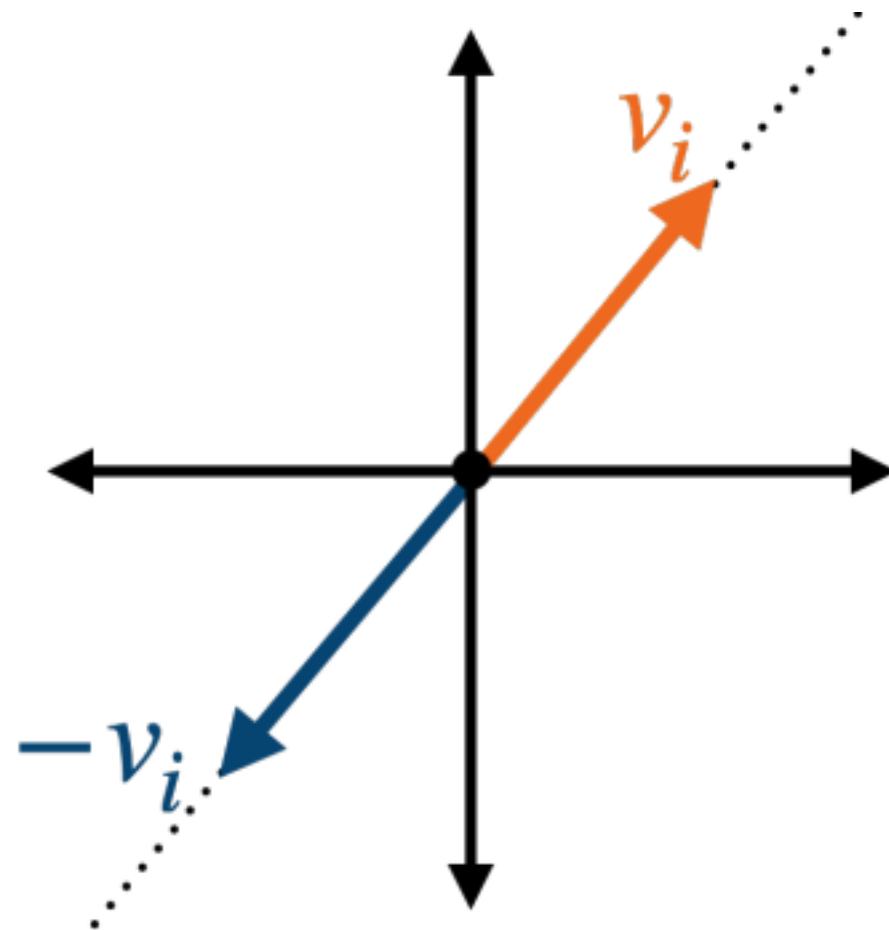
- Suppose v is a Laplacian eigenvector such that $\mathbf{L}\mathbf{x} = \lambda\mathbf{x}$
- This implies $\mathbf{L}(-\mathbf{x}) = \lambda(-\mathbf{x})$
- So $-\mathbf{x}$ is also a Laplacian eigenvector
- The sign choice is arbitrary





Eigenvector sign ambiguity

- What if we picked the other sign?
- Then the input positional encoding changes, and the model output changes
- For k eigenvectors, there are 2^k sign choices





Fixing Laplacian eigenvector sign ambiguity

- Build a neural network that is invariant to sign choices (i.e., the outputs won't change with sign choice)

$$f(\mathbf{e}_1, \dots, \mathbf{e}_k) = f(\pm \mathbf{e}_1, \dots, \pm \mathbf{e}_k)$$

- Theorem (single input): $f(\mathbf{e}_1) = f(-\mathbf{e}_1)$ (i.e. f is sign invariant) if and only if there is a function ϕ such that $f(\mathbf{e}_1) = \phi(\mathbf{e}_1) + \phi(-\mathbf{e}_1)$
- Theorem (multiple inputs): if $f(\mathbf{e}_1, \dots, \mathbf{e}_k)$ is sign invariant, then there exist functions ρ and ϕ such that

$$f(\mathbf{e}_1, \dots, \mathbf{e}_k) = \rho[\phi(\mathbf{e}_1) + \phi(-\mathbf{e}_1), \dots, \phi(\mathbf{e}_k) + \phi(-\mathbf{e}_k)]$$



SignNet

- As proposed by Lim et al.

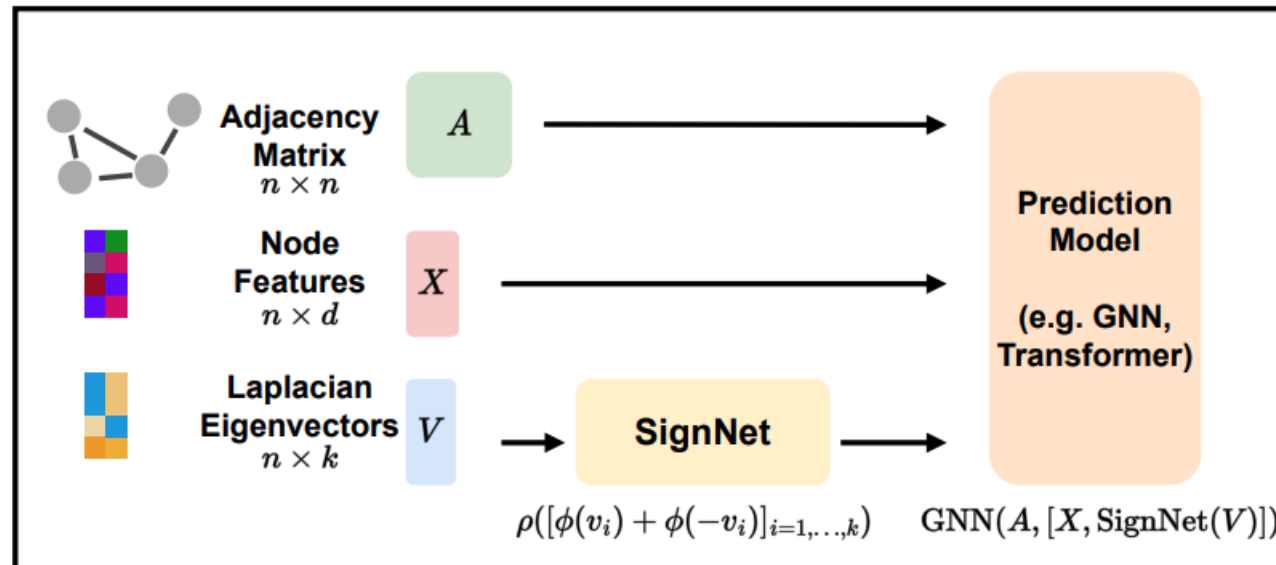
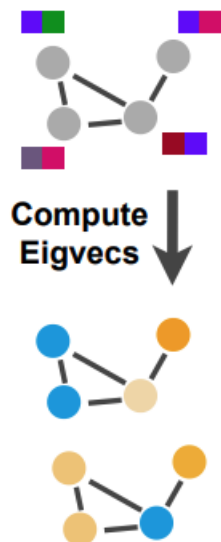
$$f(\mathbf{e}_1, \dots, \mathbf{e}_k) = \rho[\phi(\mathbf{e}_1) + \phi(-\mathbf{e}_1), \dots, \phi(\mathbf{e}_k) + \phi(-\mathbf{e}_k)]$$

where ρ and ϕ are neural networks, and k is the number of eigenvectors

- SignNet can express all sign invariant functions

SignNet in practice

1. Compute Laplacian eigenvectors
2. Create Laplacian eigenvector embeddings using SignNet
3. Concatenate SignNet embeddings with node features
4. Pass through GNN or Transformer
5. Backpropagate gradients to train SignNet and GNN or Transformer jointly

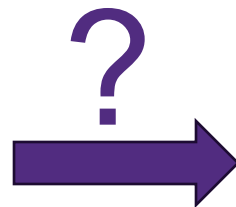




Finally, how should we address edge features?

- We need our transformer to take the following inputs:

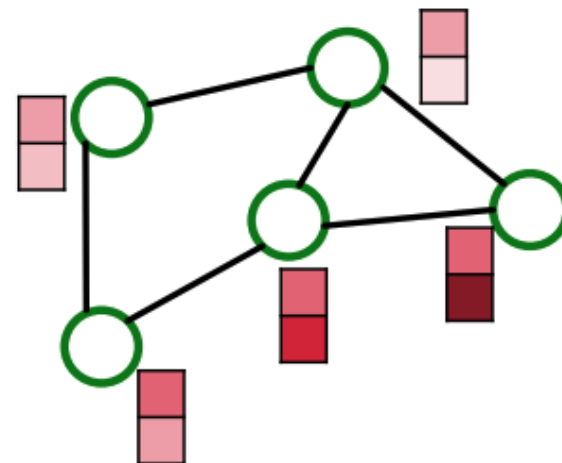
1. Node features
2. Adjacency (edge) information
3. **Edge features**



- Transformer components

- A. Token inputs
- B. Positional encoding
- C. **Self-attention**

- Different ways to do this mapping
- We'll consider
 - $1 \rightarrow A$
 - $2 \rightarrow B$
 - $3 \rightarrow C$





Edge features through modified self-attention

- Recall, the attention weights, $[a_{ij}] = QK^T$, applied to token j by token i in transformers where $Q = XW^Q$, $K = XW^K$
- Idea: adjust these to include edge features. Set $a_{ij} = a_{ij} + c_{ij}$ where c_{ij} depends on edge features



Edge features through modified self-attention

- Recall, the attention weights, $[a_{ij}] = QK^T$, applied to token j by token i in transformers where $Q = XW^Q$, $K = XW^K$
- Idea: adjust these to include edge features. Set $a_{ij} = a_{ij} + c_{ij}$ where c_{ij} depends on edge features
- Implementation
 - If there is an edge between i and j with features $\tilde{\mathbf{x}}_{ij}$ define $c_{ij} = \mathbf{w}_1^T \tilde{\mathbf{x}}_{ij}$ where $\mathbf{w}_1$ is a learnable parameter. There will be m such parameters, $\mathbf{w}_1, \dots, \mathbf{w}_m$ one for each edge
 - If there is no edge, find the shortest edge path between i and j with feature set $\{\tilde{\mathbf{x}}_{i,n_1}, \tilde{\mathbf{x}}_{n_1,n_2}, \dots, \tilde{\mathbf{x}}_{n_k,j}\}$ and define $c_{ij} = \sum_k \mathbf{w}_k^T \tilde{\mathbf{x}}_k$

Graph transformer design summary

1. Tokenization
 - a. Usually node features
 - b. Other options include subgraphs, node + edge features
2. Positional encoding – relative distances or Laplacian eigenvectors
3. Modified attention for edge features

